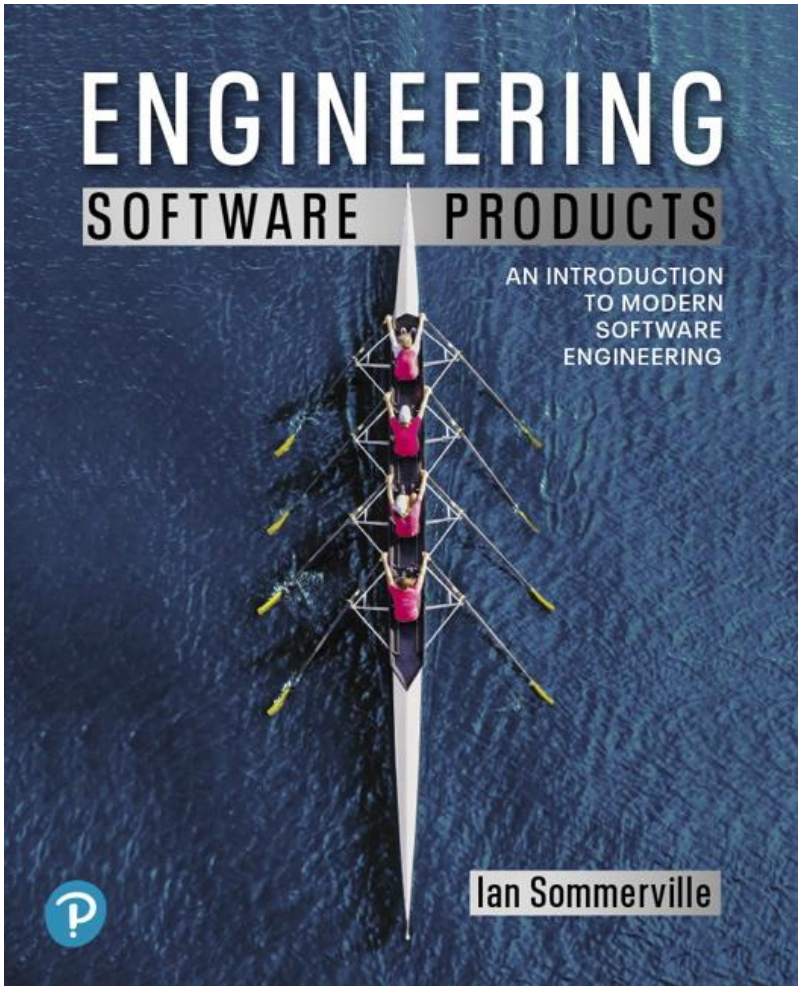
Engineering Software Products

First Edition



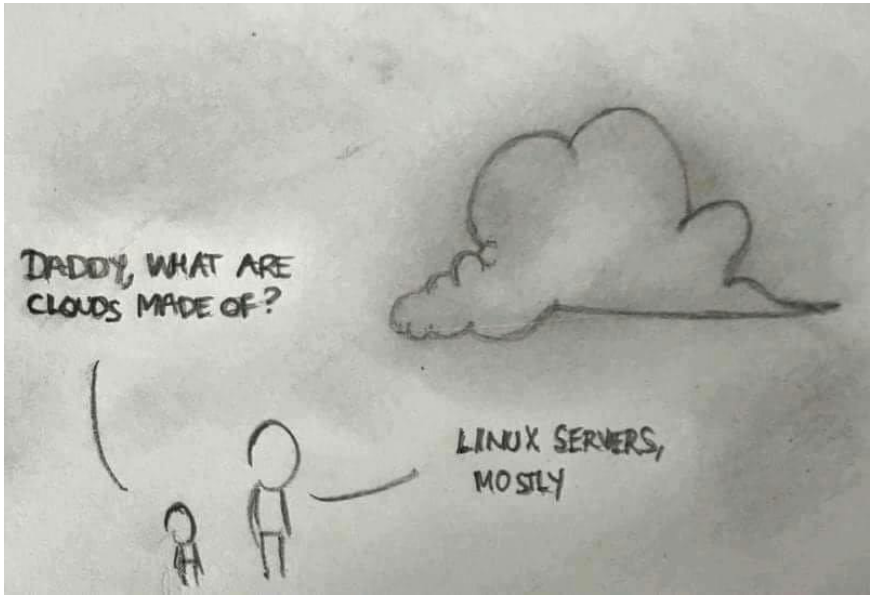
Chapter 5

Cloud-Based Software

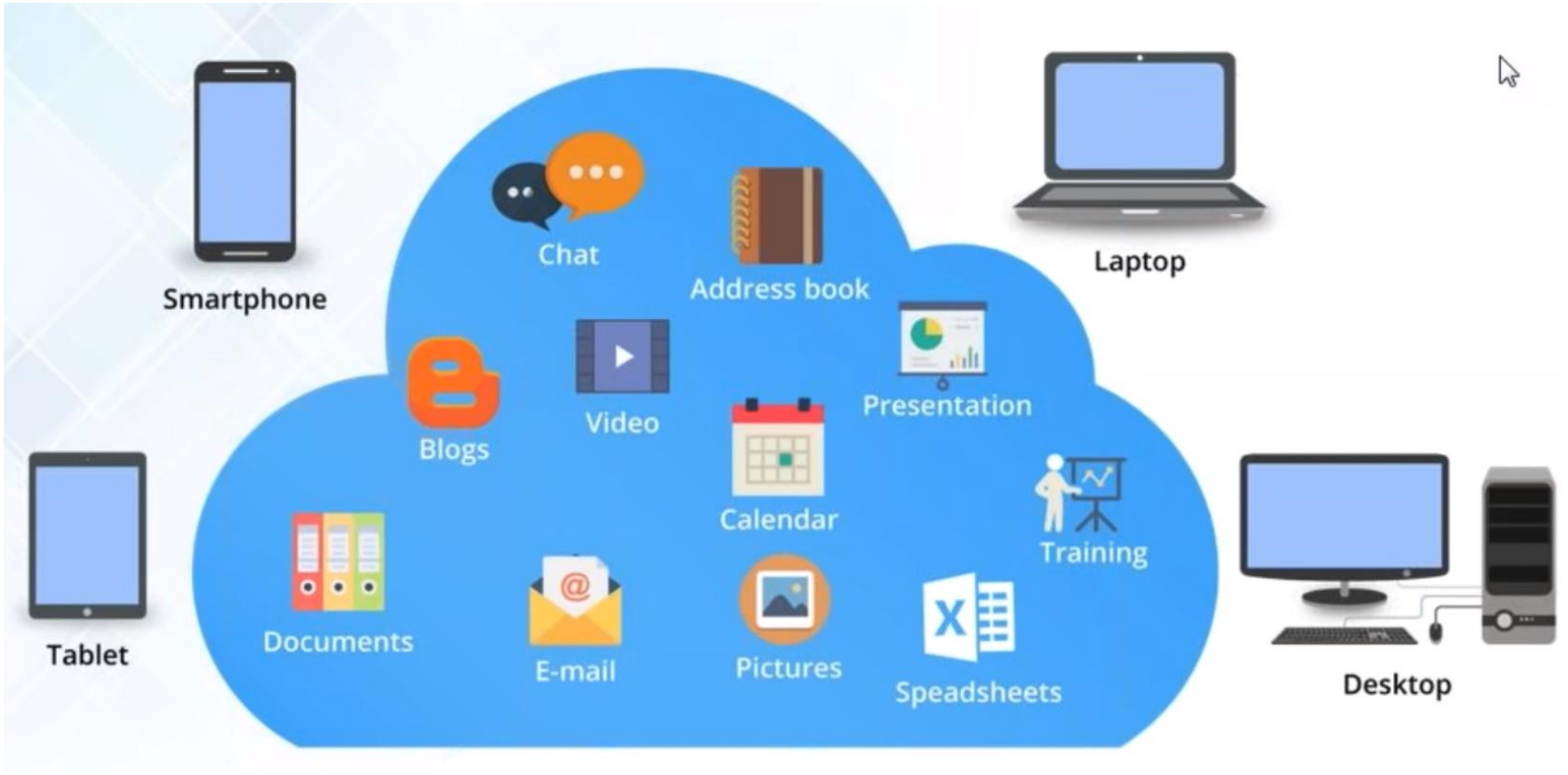
Citations

- cloud.google.com/docs
- Introduction to Cloud computing - Massimo Canonico - University of Piemonte Orientale
- Yiying Zhang
- Ali Ghodsi and Ion Stoica's Berkeley lecture notes in 2018
- Tyler Harter's HotCloud'16 OpenLambda talk
- Akshay Srivatsan, Ayelet Drazen, Jonathan Kula (Stamford)

Cloud computing is everywhere

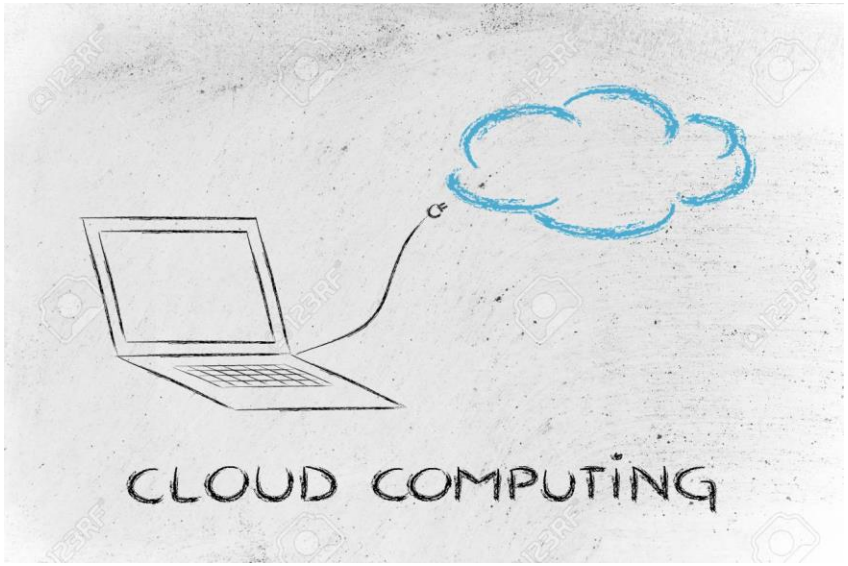


You are already a cloud user



Short history

- 14th November 1996, Huston (Texas), George Favaloro uses “Cloud” for the first time at the Compaq meeting
- Compaq bought a start-up company (NetCentric) which support the idea that data and applications will be in Internet and not in the personal PCs
- 2006 Amazon Web Service (AWS)
- 2008 Google Cloud Platform (GCP)
- 2010 Microsoft Azure
- 2020 Gaia-X: EU Cloud



What is ‘The Cloud’?

- (NIST) Cloud computing is a model for enabling ubiquitous, convenient, on-demand network access to a shared pool of configurable computing resources (e.g., networks, servers, storage, applications, and services) that can be rapidly provisioned and released with minimal management effort or service provider interaction. This cloud model is composed of:
  - 5 essential characteristics
  - 3 service models
  - 4 deployment models
- (Wikipedia) Cloud computing is the on-demand availability of computer system resources, especially data storage and computing power, without direct active management by the user. The term is generally used to describe data centers available to many users over the Internet. Large clouds, predominant today, often have functions distributed over multiple locations from central servers. If the connection to the user is relatively close, it may be designated an edge server.

‘The Cloud’ - informal definitions

- The term “Cloud computing” is misleading. As a marketing **buzzword**, it’s used to suggest that something new and better is going on, when in fact there may be **nothing new about it**.
- Cloud computing simply **increases** the **number of things that can go wrong**. And go wrong they do.
- Cloud Computing is a **business concept**, not a technological jargon dependent on "as a service model"

5 essential characteristics

1. **On-demand self-service**: consumer asks for resources (storage/processing/memory/network bandwidth) without human intervention
2. **Broad network access**: resource available over internet
3. **Resource pooling**: resources location not specified or specified at a higher level (country, region, ...)
4. **Rapid elasticity**: Capabilities can be elastically provisioned and released
5. **Measured services**: Control and optimization of the resource usage by leveraging a metering capability

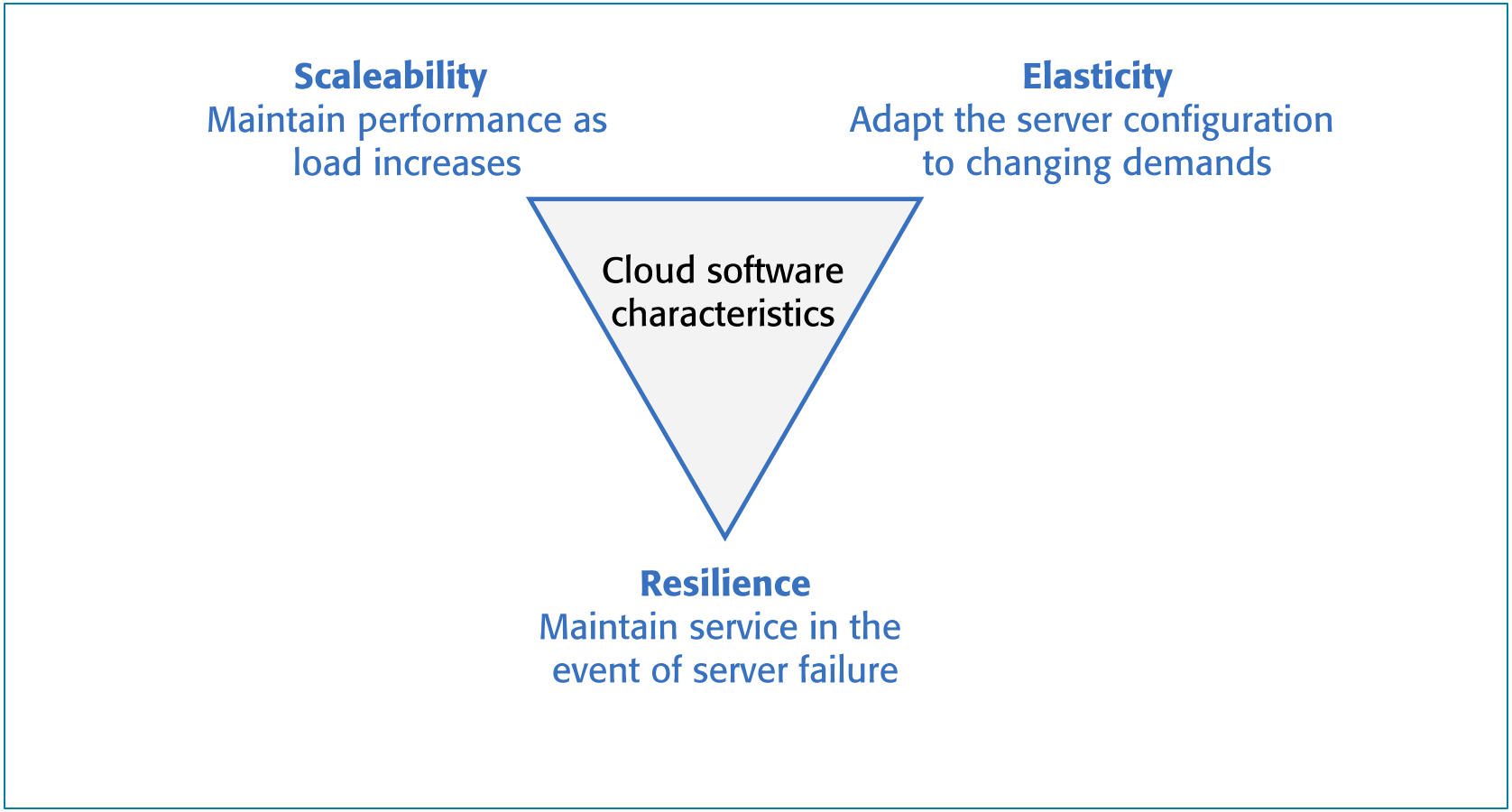
The cloud (1 of 2)

- The cloud is made up of very large number of remote servers that are offered for rent by companies that own these servers.
  - Cloud-based servers are ‘virtual servers’, which means that they are implemented in software rather than hardware.
- You can rent as many servers as you need, run your software on these servers and make them available to your customers.
  - Your customers can access these servers from their own computers or other networked devices such as a tablet or a TV.

The cloud (2 of 2)

  - Cloud servers can be started up and shut down as demand changes.
- You may rent a server and install your own software, or you may pay for access to software products that are available on the cloud.

Figure 5.1



Scalability, elasticity, and resilience

Scaleability, elasticity and resilience
(1 of 2)

- Scaleability reflects the ability of your software to cope with increasing numbers of users.
  - As the load on your software increases, your software automatically adapts so that the system performance and response time is maintained.
- Elasticity is related to scaleability but also allows for scaling-down as well as scaling-up.
  - That is, you can monitor the demand on your application and add or remove servers dynamically as the number of users change.

Scaleability, elasticity and resilience
(2 of 2)

- Resilience means that you can design your software architecture to tolerate server failures.
  - You can make several copies of your software concurrently available. If one of these fails, the others continue to provide a service.

Deployment models

- **Private cloud**: The cloud computing is provisioned for exclusive use by a single organization compromising multiple consumers
- **Community cloud**: The cloud infrastructure is provisioned for exclusive use by a specific community of consumers from organizations that have shared concerns
- **Public cloud**: The cloud infrastructure is provisioned for open use by the general public.
- **Hybrid cloud**: The cloud infrastructure is a composition of two or more distinct cloud infrastructures (private, community, or public)

Public cloud

- Owned and operated by third parties
- Give to each individual client an attractive low-cost, “Pay-as-you-go” model
- All customers share the same infrastructure pool with
  - limited configuration
  - security protections
  - availability variances
- May be larger than an enterprises cloud
- Scale seamlessly on demand

Name	VCPUS	RAM	Total Disk	Root Disk
m1.tiny	1	512 MB	1 GB	1 GB
m1.small	1	2 GB	20 GB	20 GB
m1.medium	2	4 GB	40 GB	40 GB
m1.large	4	8 GB	40 GB	40 GB
m1.xlarge	8	16 GB	40 GB	40 GB
m1.xxlarge	16	32 GB	40 GB	40 GB

Public cloud providers

Private cloud

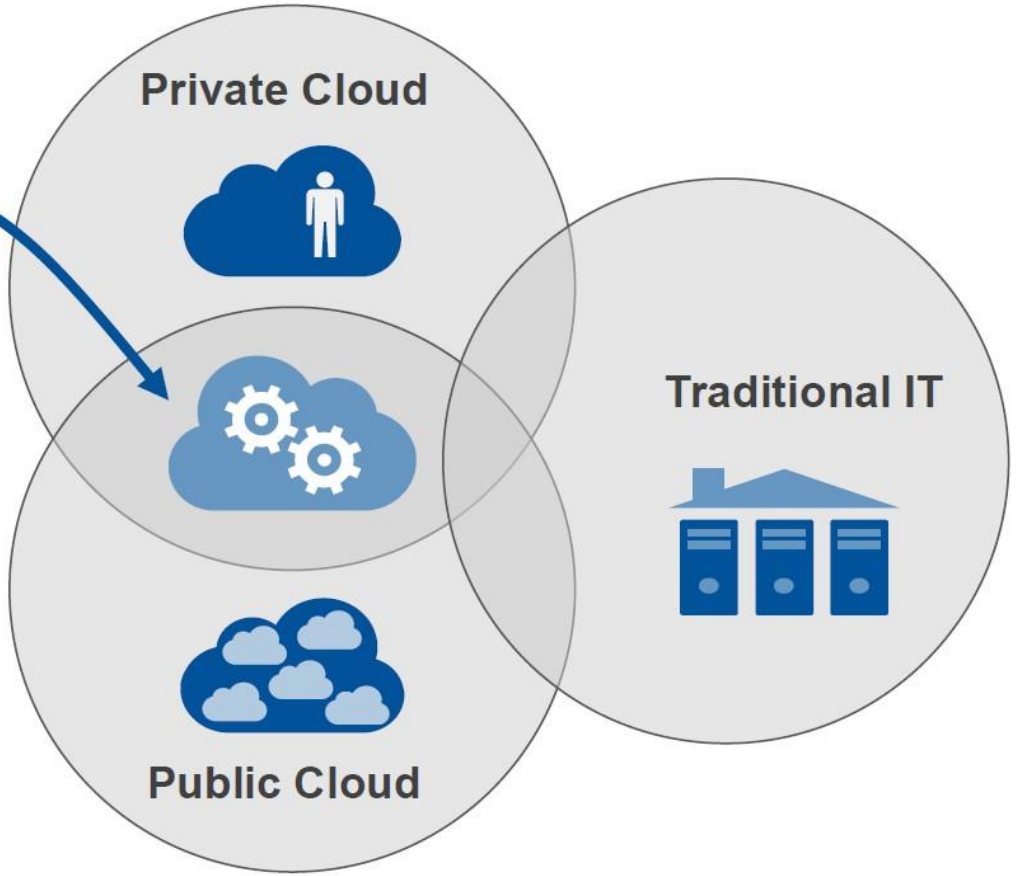
- Are built exclusively for a single enterprise
- Aim to address concerns on data security and offer greater control
- There are two options:
  - On-premise Private Cloud
  - Externally hosted Private Cloud
    - Hosted externally with a cloud provider
    - The provider facilitates an exclusive cloud environment with full guarantee of privacy
    - Best suited for enterprises that don't prefer a public cloud due to sharing of physical resources

Hybrid cloud

- Combine both public and private cloud models
- Service providers can utilize 3rd party Cloud Providers in a full or partial manner thus increasing the flexibility of computing
- The ability to augment a private cloud with the resources of a public cloud can be used to manage any unexpected surges in workload

Hybrid Cloud

A cloud computing service that is composed of some combination of private, public and community cloud services from different service providers for capacity or capability.



Cloud Servers

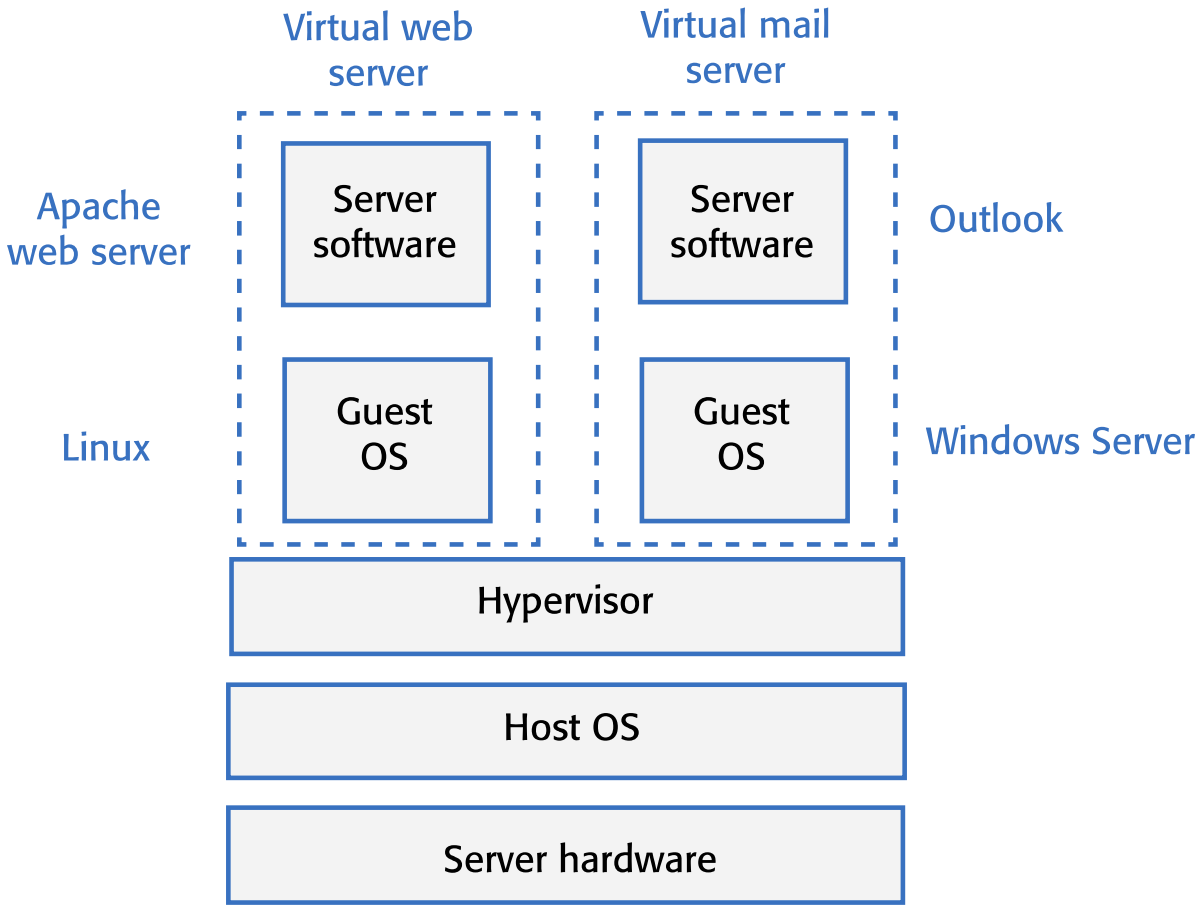
Virtual cloud servers (1 of 2)

- A virtual server runs on an underlying physical computer and is made up of an operating system plus a set of software packages that provide the server functionality required.
- A virtual server is a stand-alone system that can run on any hardware in the cloud.
  - This ‘run anywhere’ characteristic is possible because the virtual server has no external dependencies.

Virtual cloud servers (2 of 2)

- Virtual machines (VMs), running on physical server hardware, can be used to implement virtual servers.
  - A hypervisor provides hardware emulation that simulates the operation of the underlying hardware.
- If you use a virtual machine to implement virtual servers, you have exactly the same hardware platform as a physical server.

Figure 5.2



Implementing a virtual server as a virtual machine

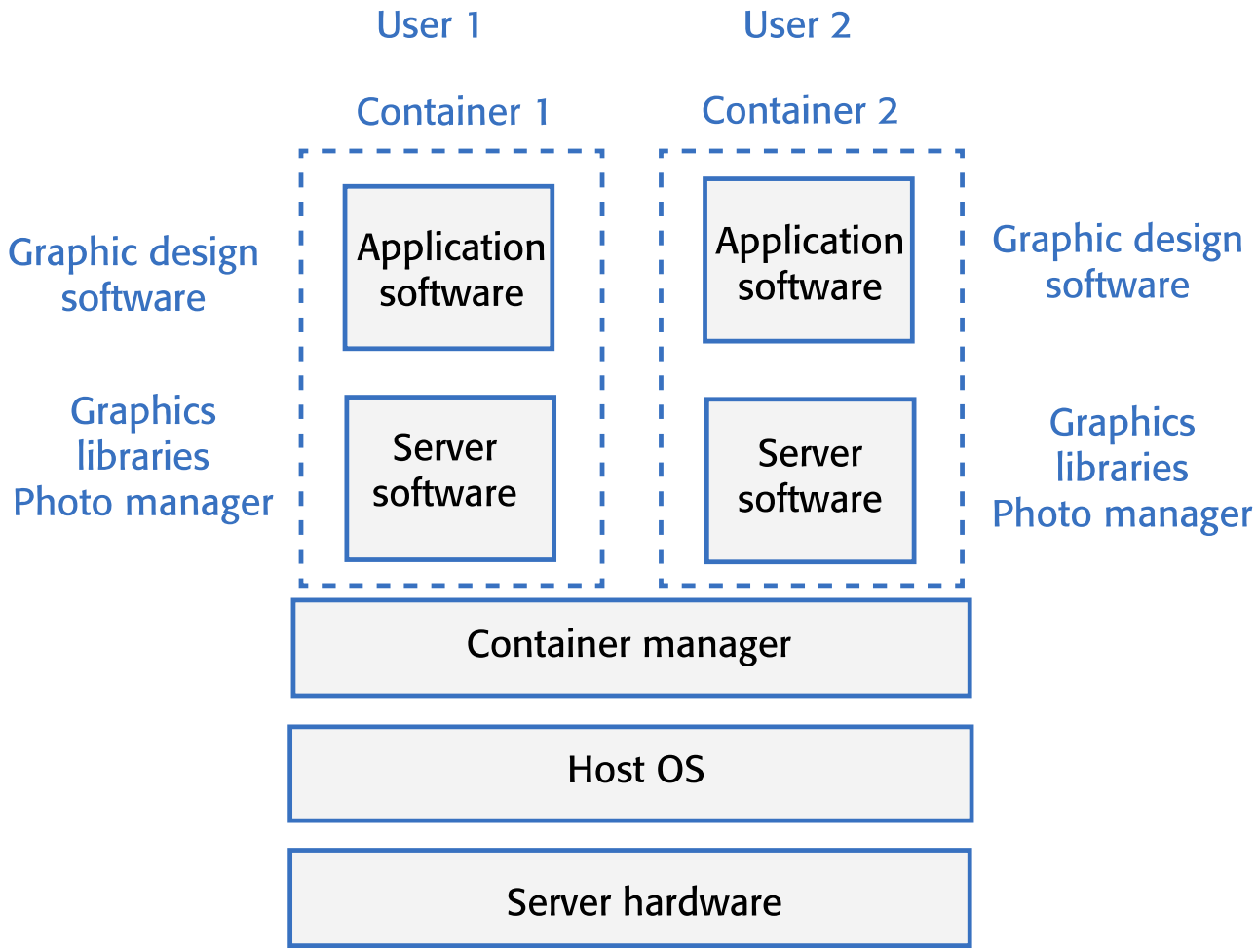
Container-based virtualization (1 of 2)

- If you are running a cloud-based system with many instances of applications or services, these all use the same operating system, you can use a simpler virtualization technology called ‘containers’.
- Using containers accelerates the process of deploying virtual servers on the cloud.
  - Containers are usually megabytes in size whereas VMs are gigabytes.
  - Containers can be started and shut down in a few seconds rather than the few minutes required for a VM.

Container-based virtualization (2 of 2)

- Containers are an operating system virtualization technology that allows independent servers to share a single operating system.
  - They are particularly useful for providing isolated application services where each user sees their own version of an application.

Figure 5.3



Using containers to provide isolated services

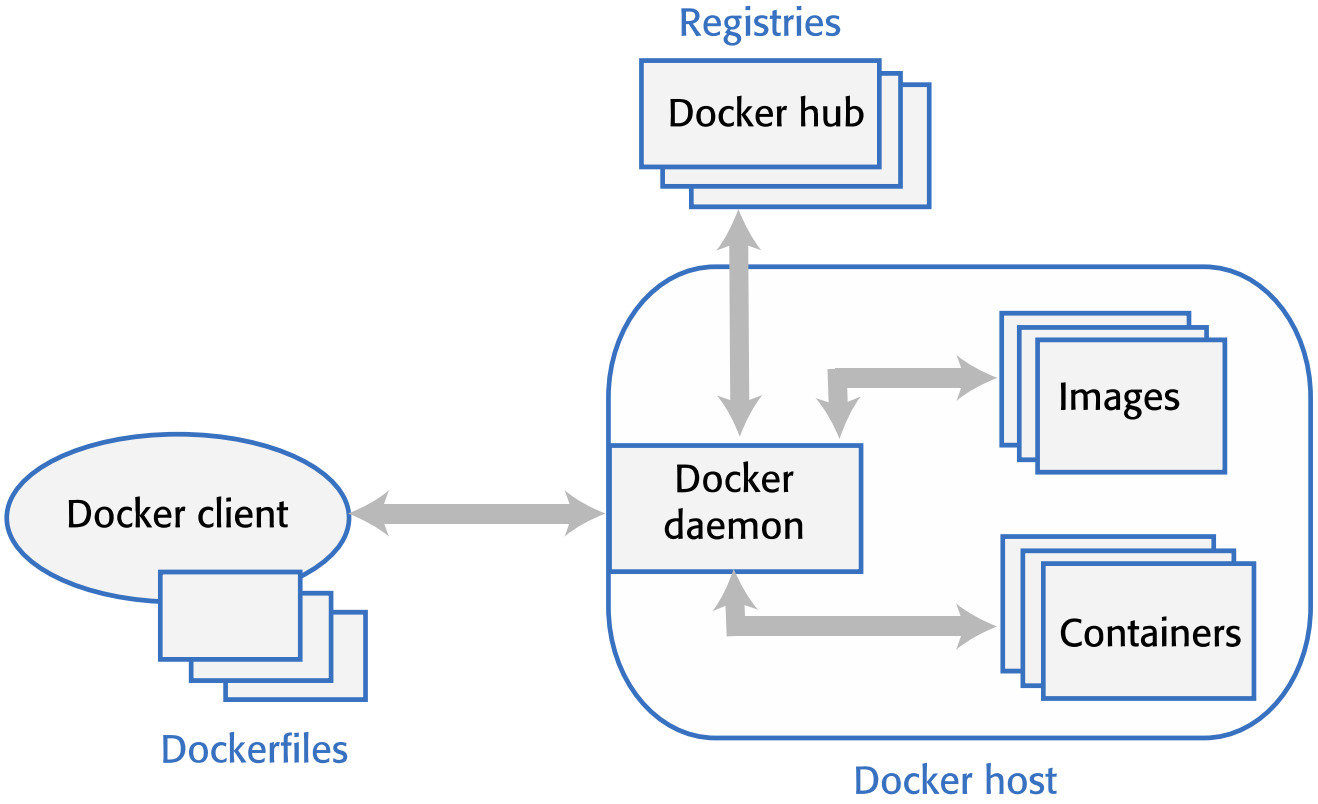
Docker (1 of 2)

- Containers were developed by Google around 2007 but containers became a mainstream technology around 2015.
- An open-source project called Docker provided a standard means of container management that is fast and easy to use.

Docker (2 of 2)

- Docker is a container management system that allows users to define the software to be included in a container as a Docker image.
- It also includes a run-time system that can create and manage containers using these Docker images.

Figure 5.4



The Docker container system

Table 5.2 The elements of the Docker container system

Element	Function
Docker daemon	This is a process that runs on a host server and is used to set up, start, stop, and monitor containers, as well as building and managing local images.
Docker client	This software is used by developers and system managers to define and control containers.
Dockerfiles	Dockerfiles define runnable applications (images) as a series of setup commands that specify the software to be included in a container. Each container must be defined by an associated Dockerfile.
Image	A Dockerfile is interpreted to create a Docker image, which is a set of directories with the specified software and data installed in the right places. Images are set up to be runnable Docker applications.
Docker hub	This is a registry of images that has been created. These may be reused to set up containers or as a starting point for defining new images.
Containers	Containers are executing images. An image is loaded into a container and the application defined by the image starts execution. Containers may be moved from server to server without modification and replicated across many servers. You can make changes to a Docker container (e.g., by modifying files) but you then must commit these changes to create a new image and restart the container.

Docker images (1 of 2)

- Docker images are directories that can be archived, shared and run on different Docker hosts. Everything that's needed to run a software system - binaries, libraries, system tools, etc. is included in the directory.

Docker images (2 of 2)

- A Docker image is a base layer, usually taken from the Docker registry, with your own software and data added as a layer on top of this.
  - The layered model means that updating Docker applications is fast and efficient. Each update to the filesystem is a layer on top of the existing system.
  - To change an application, all you have to do is to ship the changes that you have made to its image, often just a small number of files.

Benefits of containers (1 of 2)

- They solve the problem of software dependencies. You don't have to worry about the libraries and other software on the application server being different from those on your development server.
  - Instead of shipping your product as stand-alone software, you can ship a container that includes all of the support software that your product needs.
- They provide a mechanism for software portability across different clouds. Docker containers can run on any system or cloud provider where the Docker daemon is available.

Benefits of containers (2 of 2)

- They provide an efficient mechanism for implementing software services and so support the development of service-oriented architectures.
- They simplify the adoption of DevOps. This is an approach to software support where the same team are responsible for both developing and supporting operational software.

Services

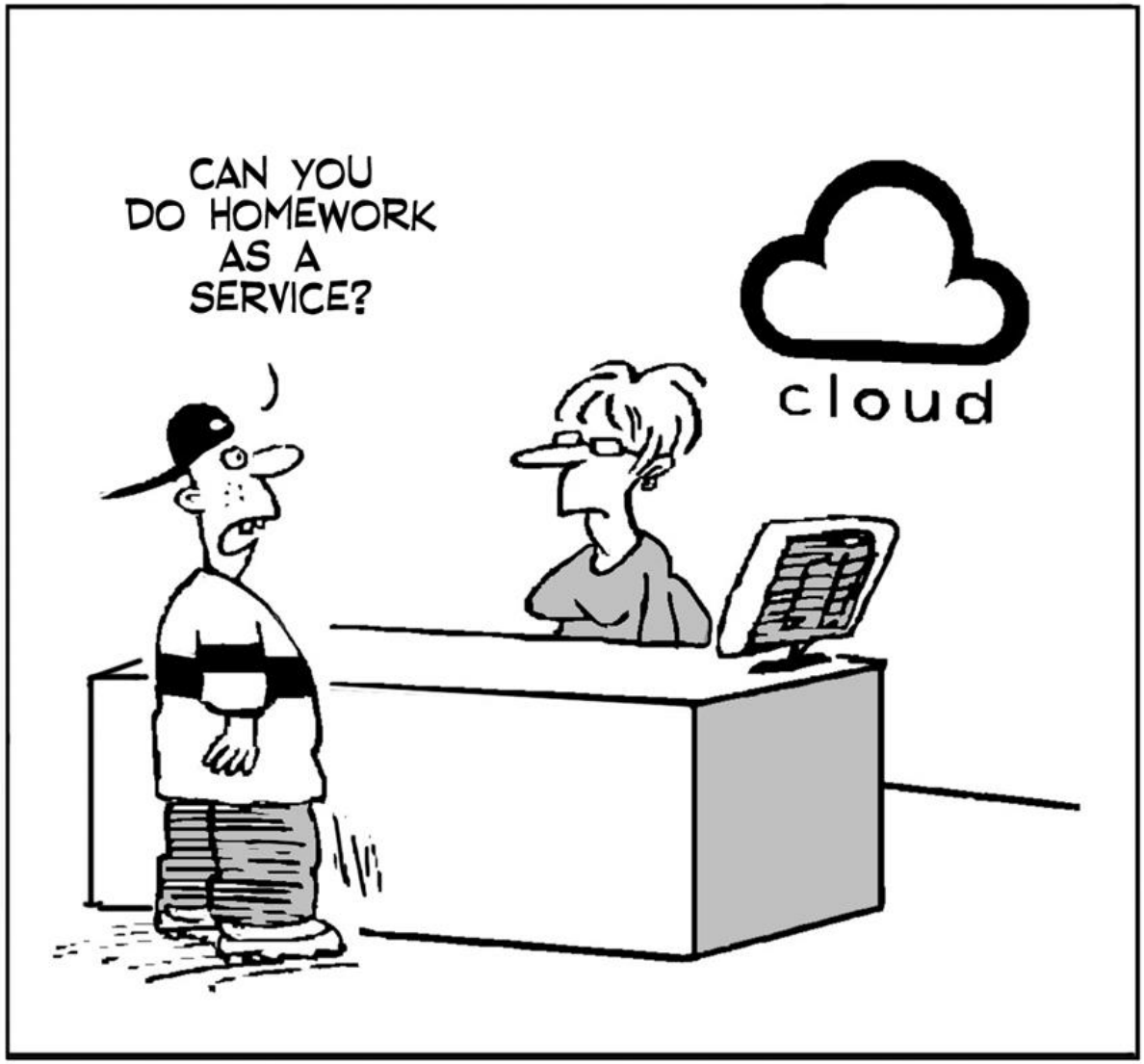
Everything as a service (1 of 2)

- The idea of a service that is rented rather than owned is fundamental to cloud computing.
- Infrastructure as a service (IaaS)
  - Cloud providers offer different kinds of infrastructure service such as a compute service, a network service and a storage service that you can use to implement virtual servers.

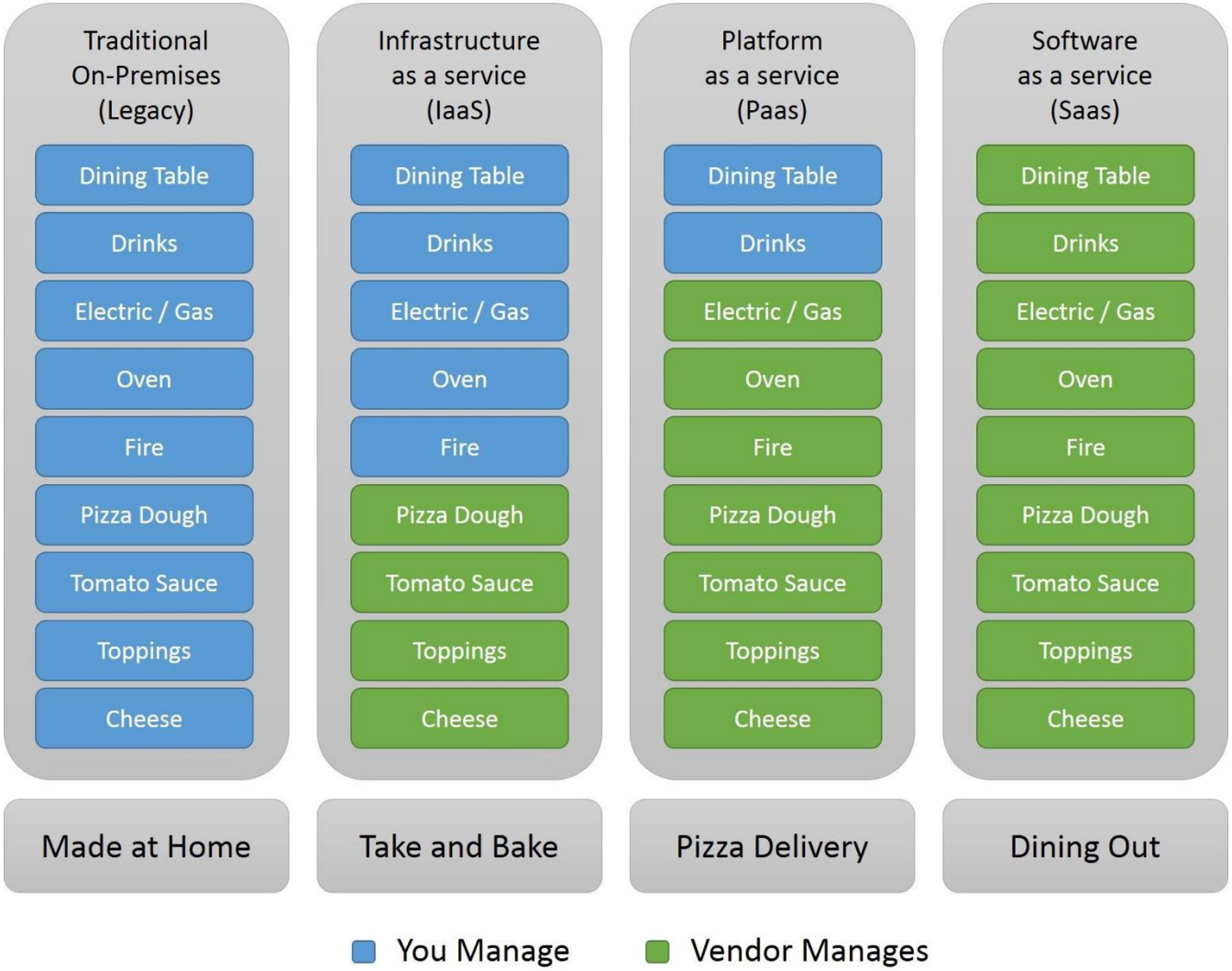
Everything as a service (2 of 2)

- Platform as a service (PaaS)
  - This is an intermediate level where you use libraries and frameworks provided by the cloud provider to implement your software. These provide access to a range of functions, including SQL and NoSQL databases.
- Software as a service (SaaS)
  - Your software product runs on the cloud and is accessed by users through a web browser or mobile app.

*aaS



Pizza as a Service



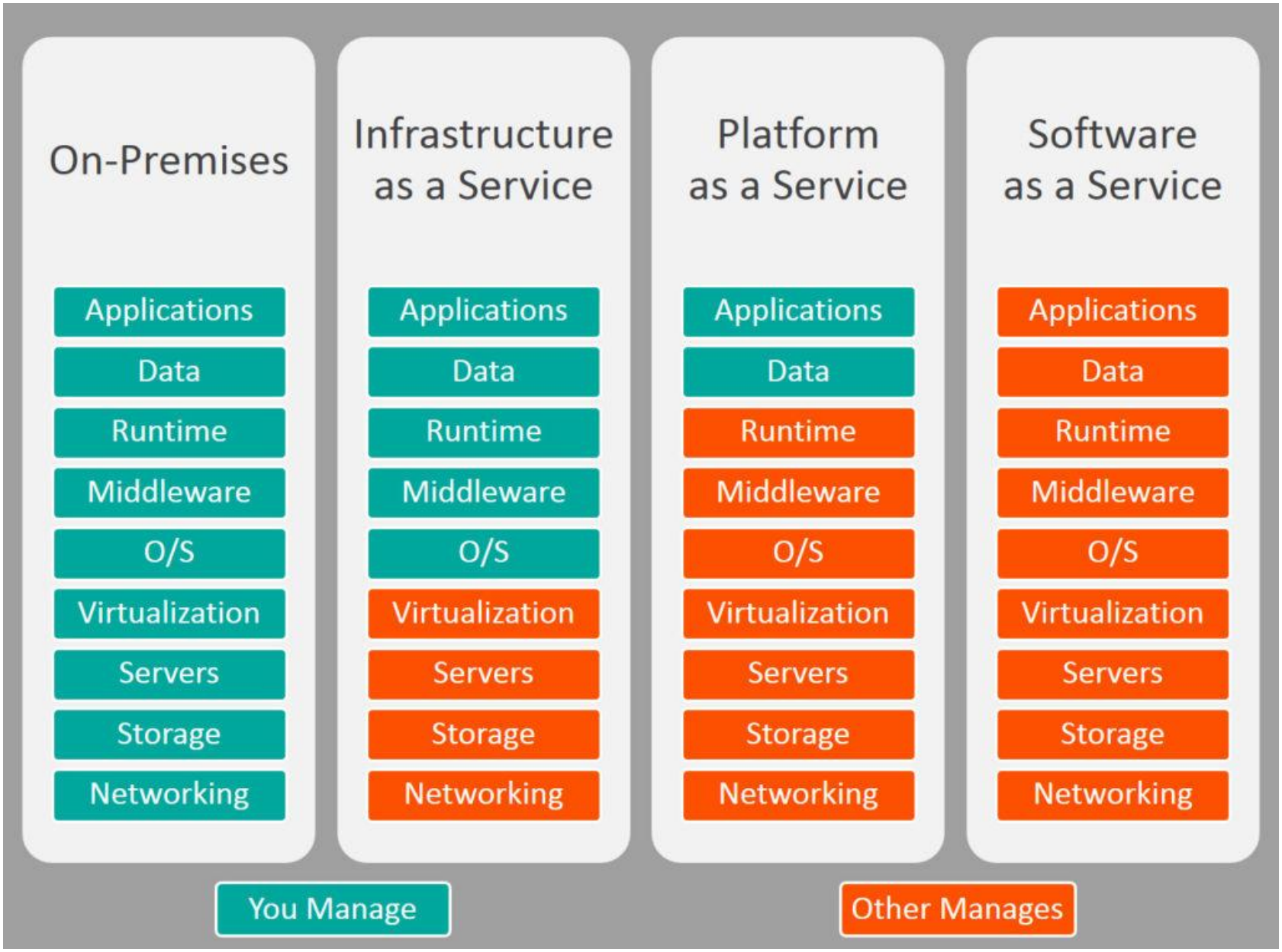
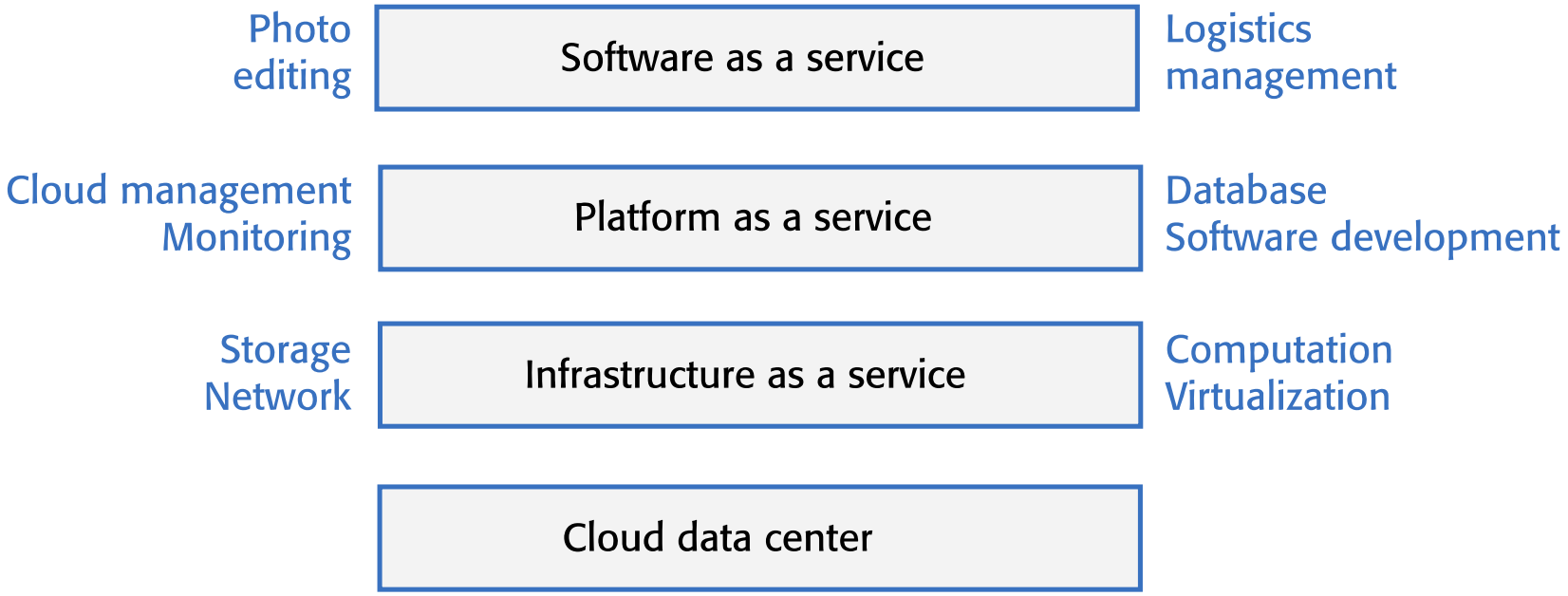
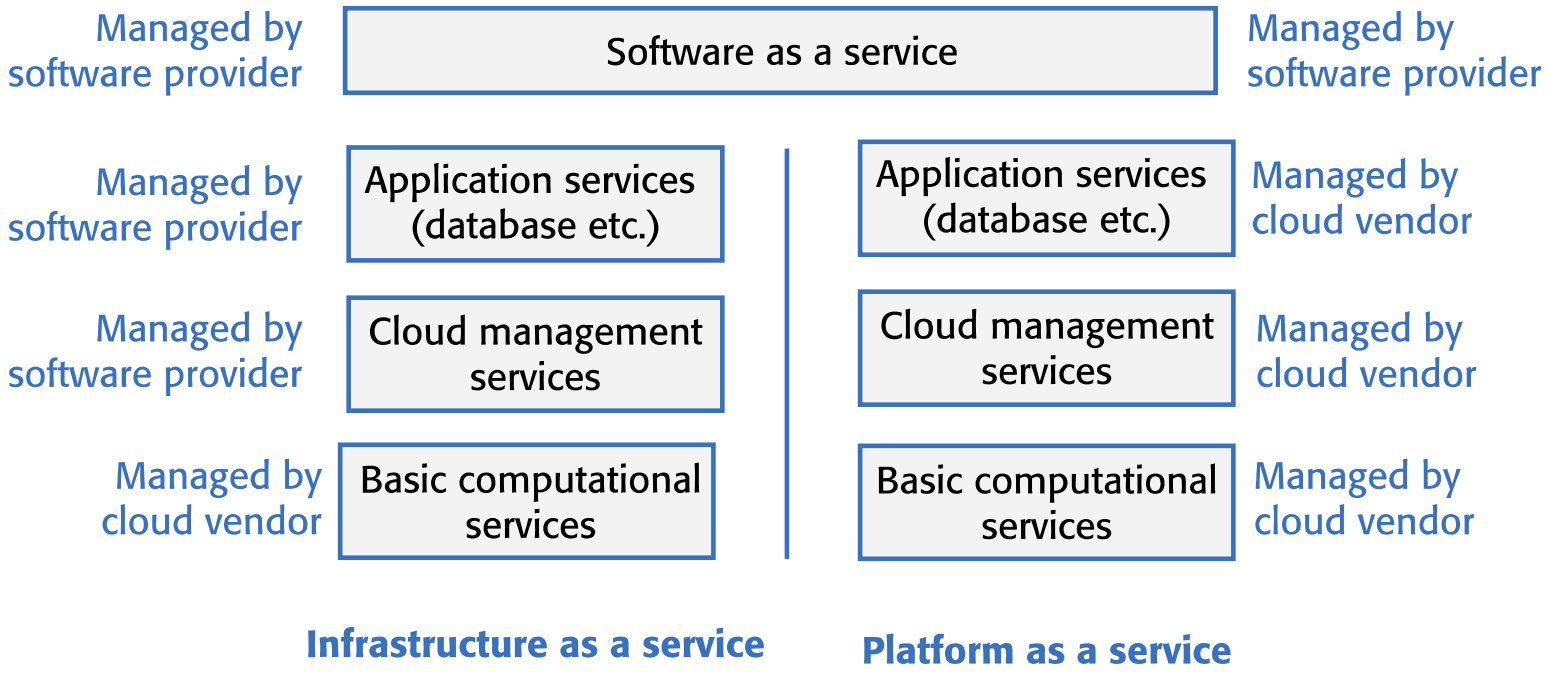


Figure 5.5



Everything as a service

Figure 5.6



Management responsibilities for SaaS, IaaS, and PaaS

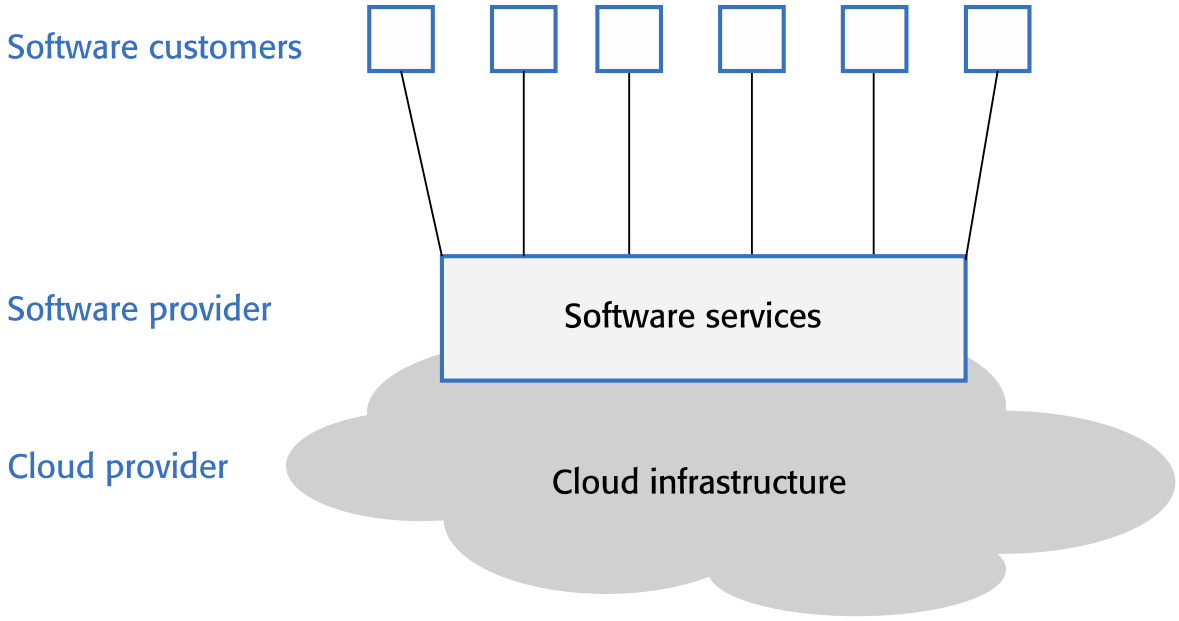
Software as a service (1 of 2)

- Increasingly, software products are being delivered as a service, rather than installed on the buyer's computers.
- If you deliver your software product as a service, you run the software on your servers, which you may rent from a cloud provider.
- Customers don't have to install software and they access the remote system through a web browser or dedicated mobile app.

Software as a service (2 of 2)

- The payment model for software as a service is usually a subscription model.
  - Users pay a monthly fee to use the software rather than buy it outright.

Figure 5.7

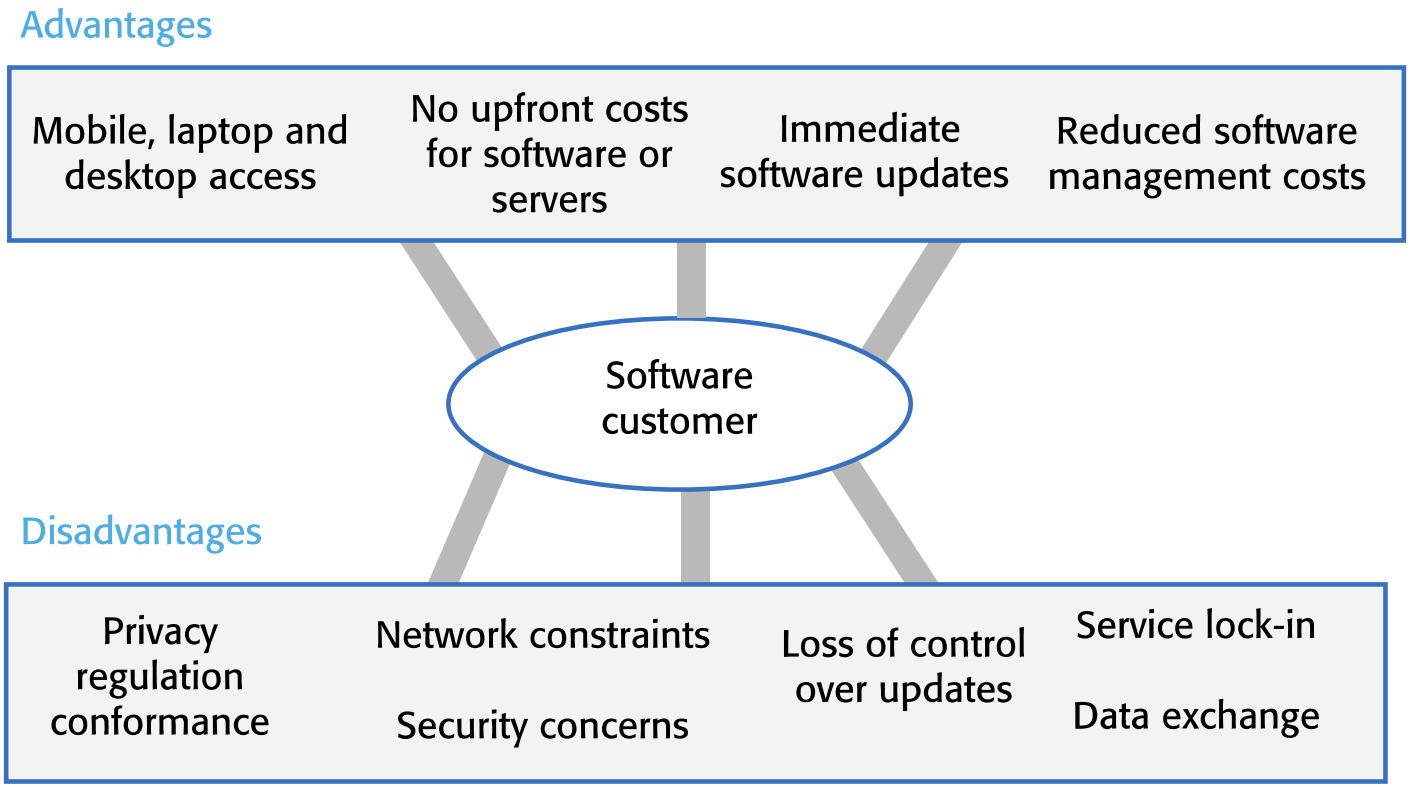


Software as a service

Table 5.3 Benefits of SaaS for software product providers

Benefit	Explanation
Cash flow	Customers either pay a regular subscription or pay as they use the software. This means you have a regular cash flow, with payments throughout the year. You don't have a situation where you have a large cash injection when products are purchased but very little income between product releases.
Update management	You are in control of updates to your product, and all customers receive the update at the same time. You avoid the issue of several versions being simultaneously used and maintained. This reduces your costs and makes it easier to maintain a consistent software code base.
Continuous deployment	You can deploy new versions of your software as soon as changes have been made and tested. This means you can fix bugs quickly so that your software reliability can continuously improve.
Payment flexibility	You can have several different payment options so that you can attract a wider range of customers. Small companies or individuals need not be discouraged by having to pay large upfront software costs.
Try before you buy	You can make early free or low-cost versions of the software available quickly with the aim of getting customer feedback on bugs and how the product could be approved.
Data collection	You can easily collect data on how the product is used and so identify areas for improvement. You may also be able to collect customer data that allow you to market other products to these customers.

Figure 5.8

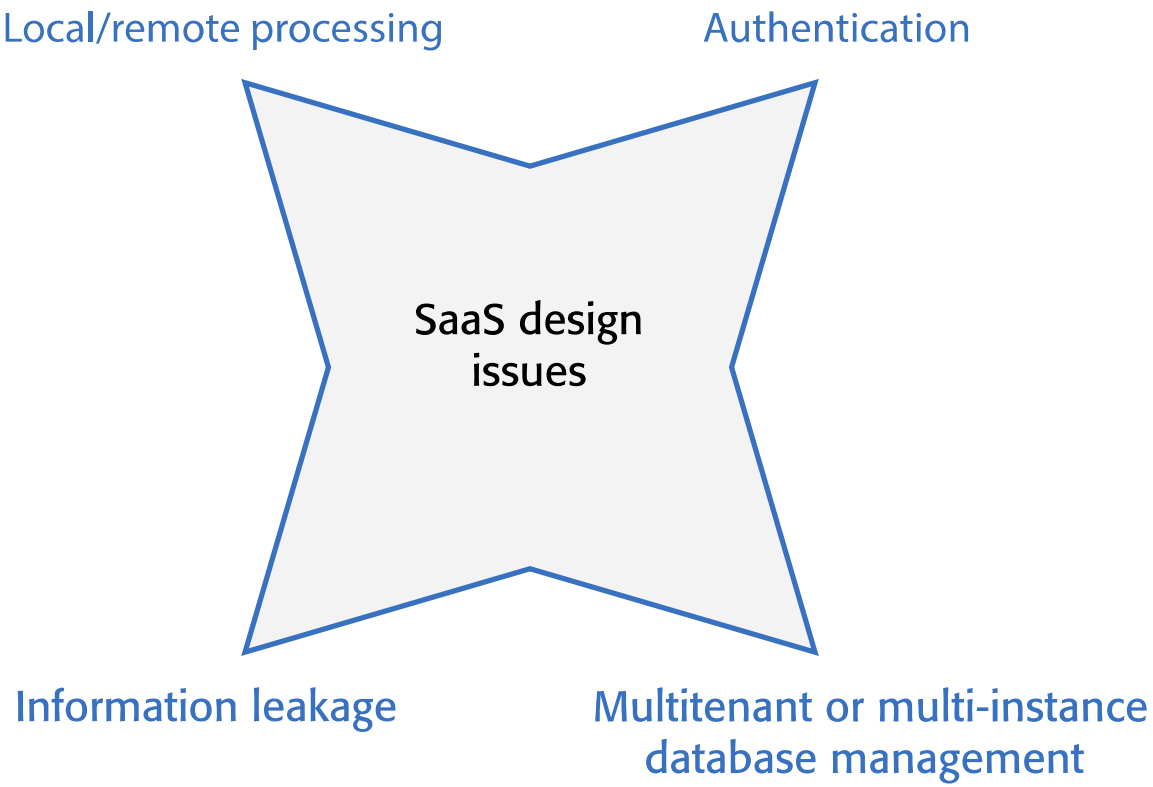


Advantages and disadvantages of SaaS for customers

Table 5.4 Data storage and management issues for SaaS

Issue	Explanation
Regulation	Some countries, such as EU countries, have strict laws on the storage of personal information. These may be incompatible with the laws and regulations of the country where the SaaS provider is based. If an SaaS provider cannot guarantee that their storage locations conform to the laws of the customer's country, businesses may be reluctant to use their product.
Data transfer	If software use involves a lot of data transfer, the software response time may be limited by the network speed. This is a problem for individuals and smaller companies who can't afford to pay for very high speed network connections.
Data security	Companies dealing with sensitive information may be unwilling to hand over the control of their data to an external software provider. As we have seen from a number of high-profile cases, even large cloud providers have had security breaches. You can't assume that they always provide better security than the customer's own servers.
Data exchange	If you need to exchange data between a cloud service and other services or local software applications, this can be difficult unless the cloud service provides an API that is accessible for external use.

Figure 5.9



Design issues for SaaS

SaaS design issues (1) (1 of 2)

- Local/remote processing
  - A software product may be designed so that some features are executed locally in the user's browser or mobile app and some on a remote server.
  - Local execution reduces network traffic and so increases user response speed. This is useful when users have a slow network connection.
  - Local processing increases the electrical power needed to run the system.

SaaS design issues (1) (2 of 2)

- Authentication
  - If you set up your own authentication system, users have to remember another set of authentication credentials.
  - Many systems allow authentication using the user's Google, Facebook or LinkedIn credentials.
  - For business products, you may need to set up a federated authentication system, which delegates authentication to the business where the user works.

SaaS design issues (2) (1 of 2)

- Information leakage
  - If you have multiple users from multiple organizations, a security risk is that information leaks from one organization to another.
  - There are a number of different ways that this can happen, so you need to be very careful in designing your security system to avoid this.

SaaS design issues (2) (2 of 2)

- Multi-tenant and multi-instance systems
  - In a multi-tenant system, all customers are served by a single instance of the system and a multitenant database.
- In a multi-instance system, a separate copy of the system and database is made available for each user.

Multi-tenant systems

- A multi-tenant database is partitioned so that customer companies have their own space and can store and access their own data.
  - There is a single database schema, defined by the SaaS provider, that is shared by all of the system's users.
  - Items in the database are tagged with a tenant identifier, representing a company that has stored data in the system. The database access software uses this tenant identifier to provide 'logical isolation', which means that users seem to be working with their own database.

Figure 5.10

Stock management					
Tenant	Key	Item	Stock	Supplier	Ordered
T516	100	Widg 1	27	S13	2017/2/12
T632	100	Obj 1	5	S13	2017/1/11
T973	100	Thing 1	241	S13	2017/2/7
T516	110	Widg 2	14	S13	2017/2/2
T516	120	Widg 3	17	S13	2017/1/24
T973	100	Thing 2	132	S26	2017/2/12

An example of a multi-tenant database

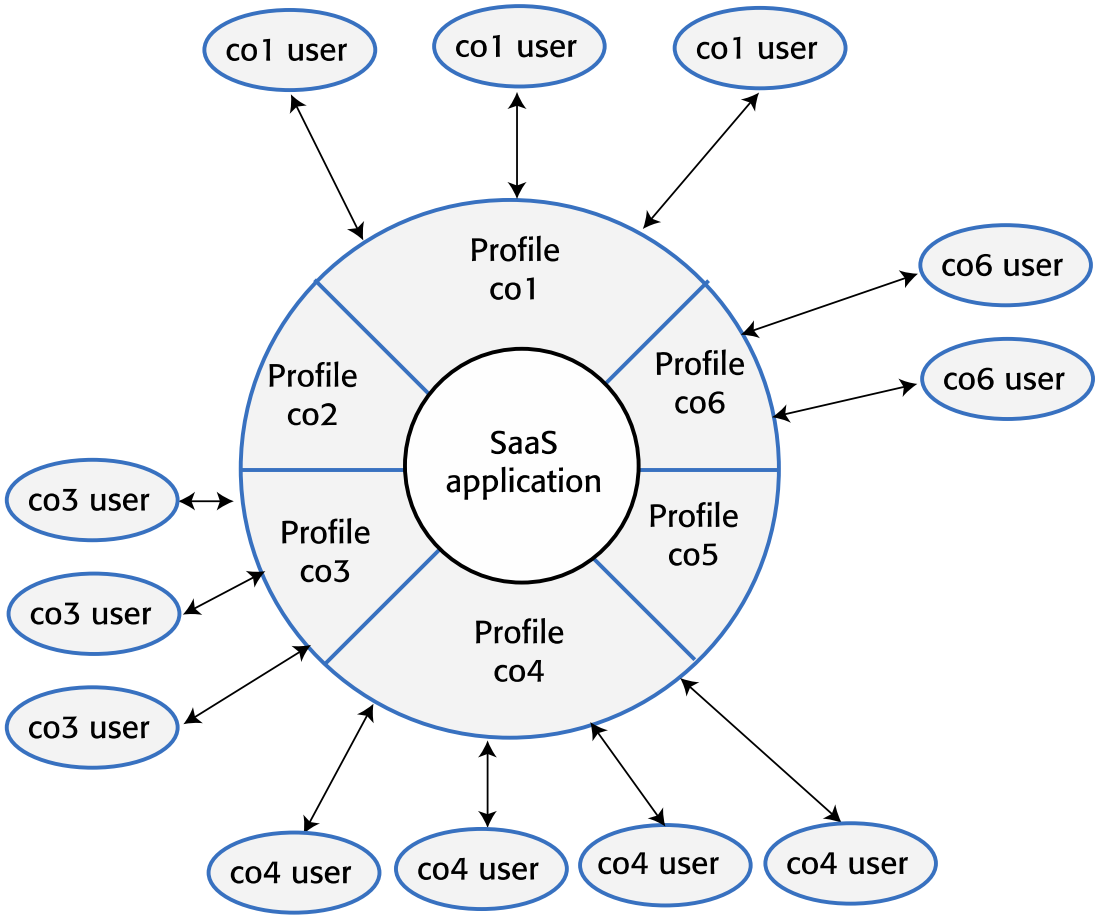
Table 5.5 Advantages and disadvantages of multi-tenant databases

Advantages	Disadvantages
Resource utilization The SaaS provider has control of all the resources used by the software and can optimize the software to make effective use of these resources.	**Inflexibility** Customers must all use the same database schema with limited scope for adapting this schema to individual needs. I explain possible database adaptations later in this section.
Security Multi-tenant databases have to be designed for security because the data for all customers are held in the same database. They are, therefore, likely to have fewer security vulnerabilities than standard database products. Security management is also simplified as there is only a single copy of the database software to be patched if a security vulnerability is discovered.	**Security** As data for all customers are maintained in the same database, there is a theoretical possibility that data will leak from one customer to another. In fact, there are very few instances of this happening. More seriously, perhaps, if there is a database security breach, then it affects all customers.
Update management It is easier to update a single instance of software rather than multiple instances. Updates are delivered to all customers at the same time so all use the latest version of the software.	**Complexity** Multi-tenant systems are usually more complex than multi-instance systems because of the need to manage many users. There is, therefore, an increased likelihood of bugs in the database software.

Table 5.6 Possible customizations for SaaS

Customization	Business need
Authentication	Businesses may want users to authenticate using their business credentials rather than the account credentials set up by the software provider. I explain in Chapter 7 how federated authentication makes this possible.
Branding	Businesses may want a user interface that is branded to reflect their own organization.
Business rules	Businesses may want to be able to define their own business rules and workflows that apply to their own data.
Data schemas	Businesses may want to be able to extend the standard data model used in the system database to meet their own business needs.
Access control	Businesses may want to be able to define their own access control model that sets out the data that specific users or user groups can access and the allowed operations on that data.

Figure 5.11



User profiles for SaaS access

Figure 5.12

Stock management								
Tenant	Key	Item	Stock	Supplier	Ordered	Ext 1	Ext 2	Ext 3
T516	100	Widg 1	27	S13	2017/2/12			
T632	100	Obj 1	5	S13	2017/1/11			
T973	100	Thing 1	241	S13	2017/2/7			
T516	110	Widg 2	14	S13	2017/2/2			
T516	120	Widg 3	17	S13	2017/1/24			
T973	100	Thing 2	132	S26	2017/2/12			

Database extensibility using additional fields

Adding fields to extend the database
(1 of 2)

- You add some extra columns to each database table and define a customer profile that maps the column names that the customer wants to these extra columns. However:
  - It is difficult to know how many extra columns you should include. If you have too few, customers will find that there aren't enough for what they need to do.
  - If you cater for customers who need a lot of extra columns, however, you will find that most customers don't use them, so you will have a lot of wasted space in your database.

Adding fields to extend the database
(2 of 2)

- Different customers are likely to need different types of columns.
  - For example, some customers may wish to have columns whose items are string types, others may wish to have columns that are integers.
  - You can get around this by maintaining everything as strings. However, this means that either you or your customer have to provide conversion software to create items of the correct type.

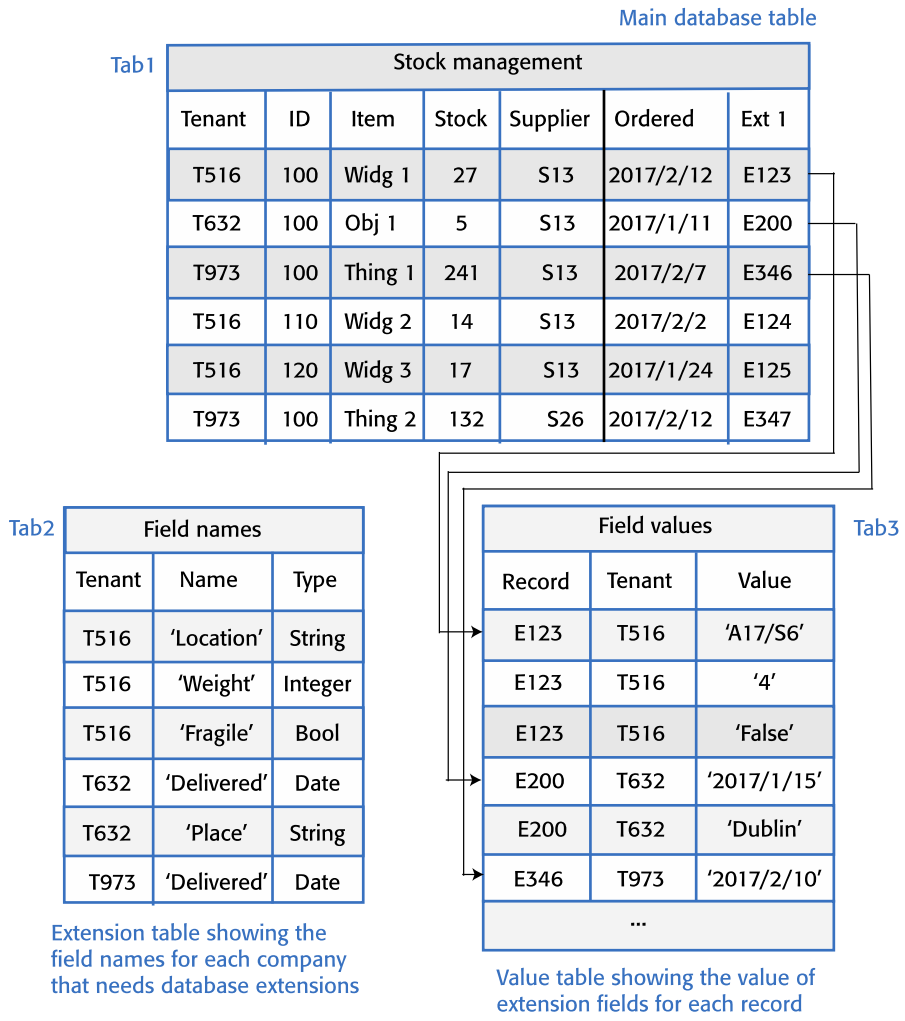
Extending a database using tables (1 of 2)

- An alternative approach to database extensibility is to allow customers to add any number of additional fields and to define the names, types and values of these fields.
- The names and types of these values are held in a separate table, accessed using the tenant identifier.

Extending a database using tables (2 of 2)

- Unfortunately, using tables in this way adds complexity to the database management software.
  - Extra tables must be managed and information from them integrated into the database.

Figure 5.13



Database extensibility using tables

Database security (1 of 2)

- Information from all customers is stored in the same database in a multi-multi-tenant system so a software bug or an attack could lead to the data of some or all customers being exposed to others.
- Key security issues are multilevel access control and encryption.
  - Multilevel access control means that access to data must be controlled at both the organizational level and the individual level.

Database security (2 of 2)

  - You need to have organizational level access control to ensure that any database operations only act on that organization's data. The individual user accessing the data should also have their own access permissions.

- Encryption of data in a multitenant database reassures corporate users that their data cannot be viewed by people from other companies if some kind of system failure occurs.

Multi-instance databases (1 of 3)

- Multi-instance systems are SaaS systems where each customer has its own system that is adapted to its needs, including its own database and security controls.
- Multi-instance, cloud-based systems are conceptually simpler than multi-tenant systems and avoid security concerns such as data leakage from one organization to another.

Multi-instance databases (2 of 3)

- There are two types of multi-instance system:
  - VM-based multi-instance systems are multi-instance systems where the software instance and database for each customer runs in its own virtual machine. All users from the same customer may access the shared system database.
  - Container-based multi-instance systems* These are multi-instance systems where each user has an isolated version of the software and database running in a set of containers.

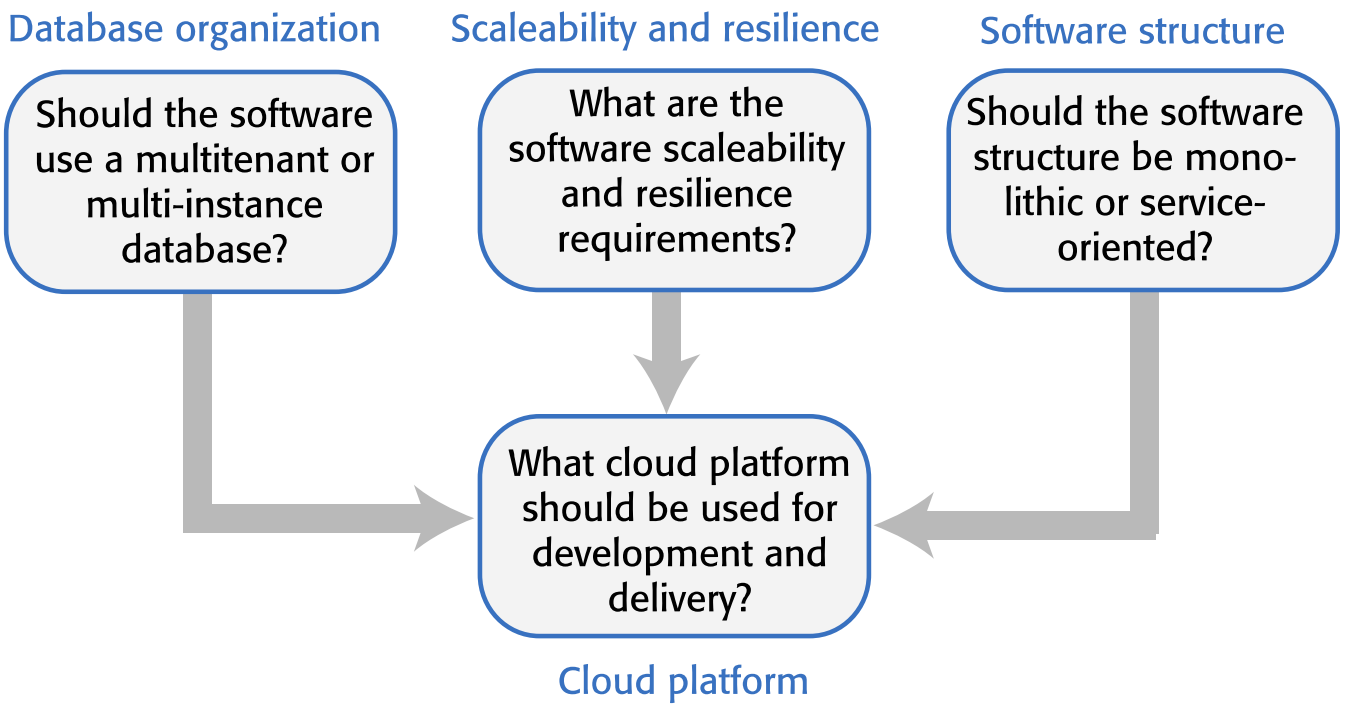
Multi-instance databases (3 of 3)

- This approach is suited to products in which users mostly work independently, with relatively little data sharing. Therefore, it is best used for software that serves individuals rather than business customers or for business products that are not data-intensive.

Table 5.7 Advantages and disadvantages of multi-instance databases

Advantages	Disadvantages
Flexibility Each instance of the software can be tailored and adapted to a customer's needs. Customers may use completely different database schemas and it is straightforward to transfer data from a customer database to the product database.	**Cost** It is more expensive to use multi-instance systems because of the costs of renting many VMs in the cloud and the costs of managing multiple systems. Because of the slow startup time, VMs may have to be rented and kept running continuously, even if there is very little demand for the service.
Security Each customer has its own database so there is no possibility of data leakage from one customer to another.	**Update management** Many instances have to be updated so updates are more complex, especially if instances have been tailored to specific customer needs.
Scalability Instances of the system can be scaled according to the needs of individual customers. For example, some customers may require more powerful servers than others.	
Resilience If a software failure occurs, this will probably affect only a single customer. Other customers can continue working as normal.	

Figure 5.14



Architectural decisions for cloud software engineering

Table 5.8 Questions to ask when choosing a database organization

Factor	Key questions
Target customers	Do customers require different database schemas and database personalization? Do customers have security concerns about database sharing? If so, use a multi-instance database.
Transaction requirements	Is it critical that your products support ACID transactions where the data are guaranteed to be consistent at all times? If so, use a multi-tenant database or a VM-based multi-instance database.
Database size and connectivity	How large is the typical database used by customers? How many relationships are there between database items? A multi-tenant model is usually best for very large databases, as you can focus effort on optimizing performance.
Database interoperability	Will customers wish to transfer information from existing databases? What are the differences in schemas between these and a possible multi-tenant database? What software support will they expect to do the data transfer? If customers have many different schemas, a multi-instance database should be used.
System structure	Are you using a service-oriented architecture for your system? Can customer databases be split into a set of individual service databases? If so, use containerized, multi-instance databases.

Scalability and resilience (1 of 2)

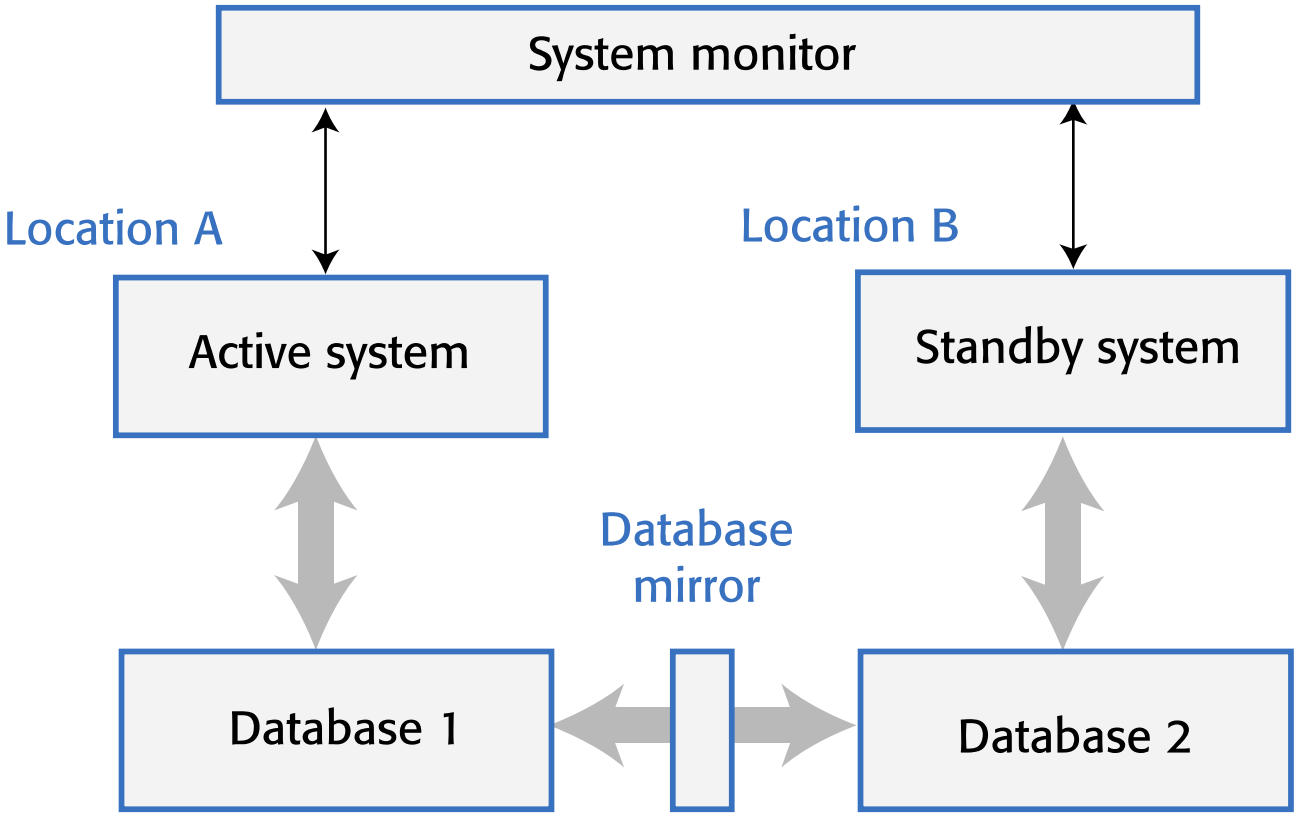
- The scaleability of a system reflects its ability to adapt automatically to changes in the load on that system.
- The resilience of a system reflects its ability to continue to deliver critical services in the event of system failure or malicious system use.
- You achieve scaleability in a system by making it possible to add new virtual servers (scaling-out) or increase the power of a system server (scaling-up) in response to increasing load.

Scalability and resilience (2 of 2)

  - In cloud-based systems, scaling-out rather than scaling-up is the normal approach used. Your software has to be organized so that individual software components can be replicated and run in parallel.

- To achieve resilience, you need to be able to restart your software quickly after a hardware or software failure.

Figure 5.15



Using a standby system to provide resilience

Resilience (1 of 2)

- Resilience relies on redundancy:
  - Replicas of the software and data are maintained in different locations.
  - Database updates are mirrored so that the standby database is a working copy of the operational database.
  - A system monitor continually checks the system status. It can switch to the standby system automatically if the operational system fails.

Resilience (2 of 2)

- You should use redundant virtual servers that are not hosted on the same physical computer and locate servers in different locations.
  - Ideally, these servers should be located in different data centers.
  - If a physical server fails or if there is a wider data center failure, then operation can be switched automatically to the software copies elsewhere.

System structure (1 of 2)

- An object-oriented approach to software engineering has been that been extensively used for the development of client-server systems built around a shared database.
- The system itself is, logically, a monolithic system with distribution across multiple servers running large software components. The traditional multi-tier client server architecture is based on this distributed system model.

System structure (2 of 2)

- The alternative to a monolithic approach to software organization is a service-oriented approach where the system is decomposed into fine-grain, stateless services.
  - Because it is stateless, each service is independent and can be replicated, distributed and migrated from one server to another.
  - The service-oriented approach is particularly suitable for cloud-based software, with services deployed in containers.

Cloud platform (1 of 2)

- Cloud platforms include general-purpose clouds such as Amazon Web Services or lesser known platforms oriented around a specific application, such as the SAP Cloud Platform. There are also smaller national providers that provide more limited services but who may be more willing to adapt their services to the needs of different customers.

Cloud platform (2 of 2)

- There is no ‘best’ platform and you should choose a cloud provider based on your background and experience, the type of product that you are developing and the expectations of your customers.
- You need to consider both technical issues and business issues when choosing a cloud platform for your product.

Cloud Pros

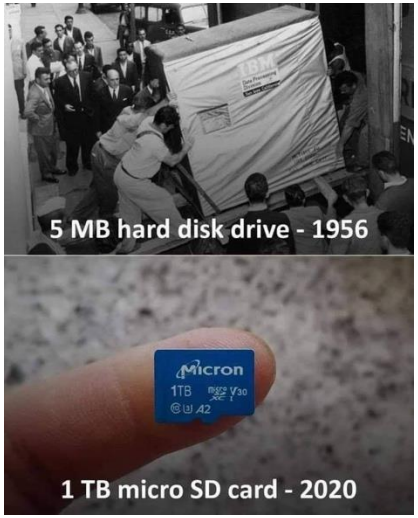
- Any ideas?

Reduced cost

- The billing model is pay as you go
- The infrastructure is not purchased thus lowering maintenance
- Initial expense and recurring expenses are much lower than traditional computing

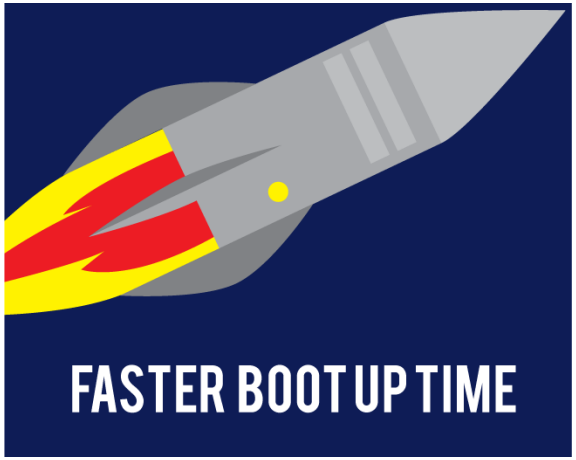
Increase storage

- With the massive infrastructure that is offered by Cloud providers today, storage and maintenance of large volumes of data is achievable
- Sudden workload spikes are also managed effectively and efficiently, since the cloud can scale dynamically



Faster startup

- You don’t have to wait for hardware to be delivered before you can start work. Using the cloud, you can have servers up and running in a few minutes.

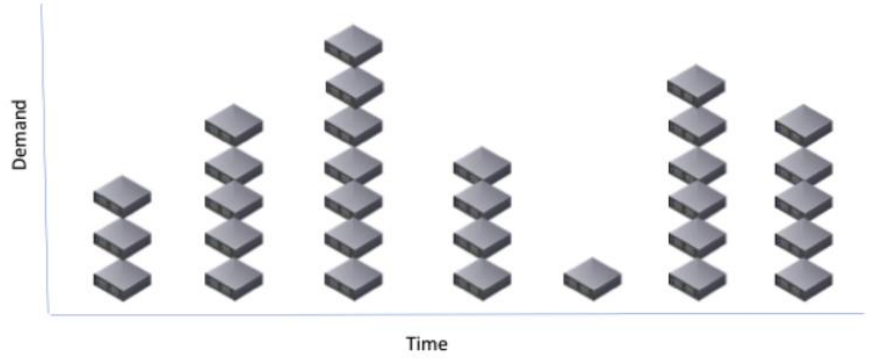


Support for distributed development

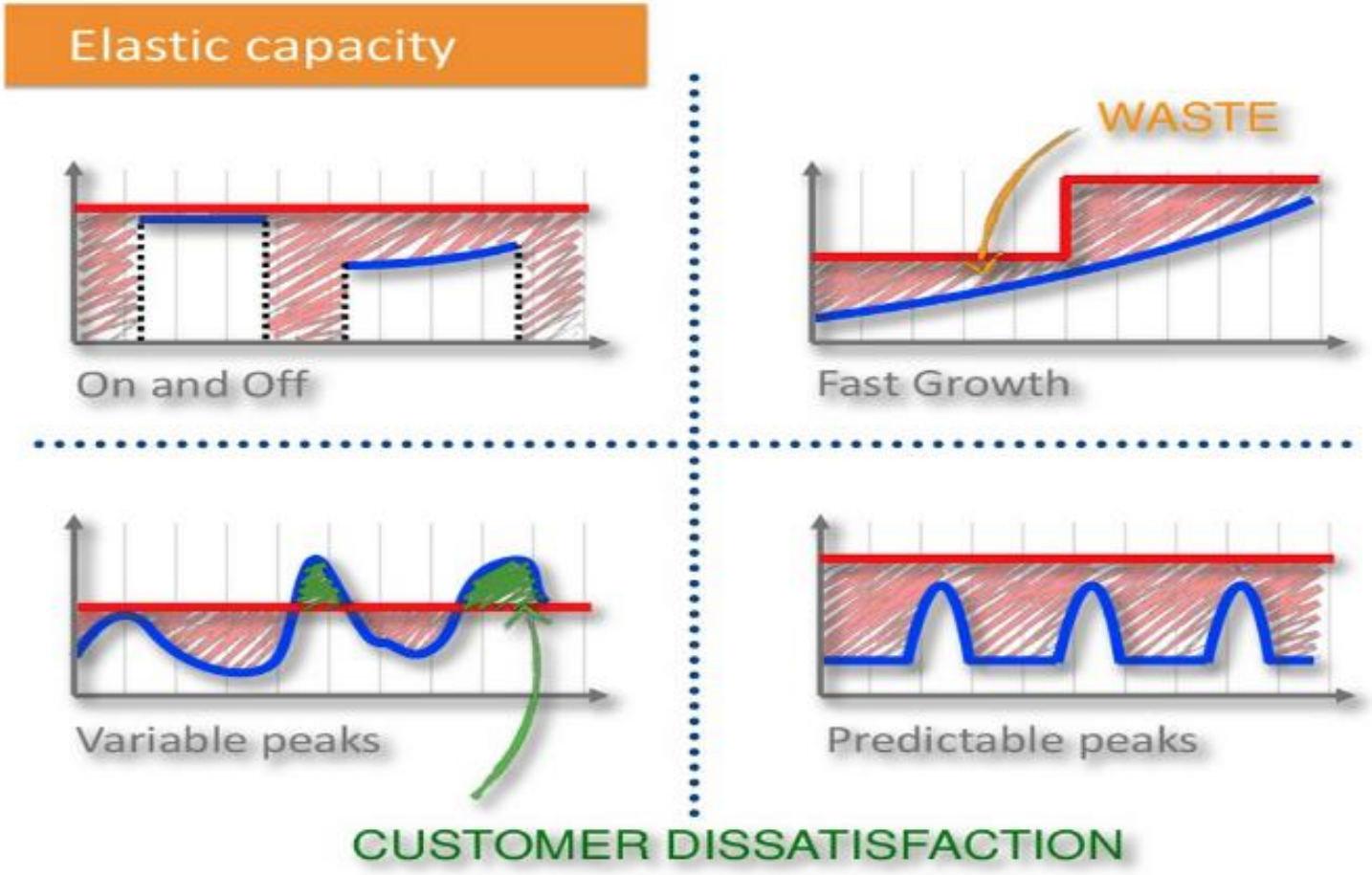
- If you have a distributed development team, working from different locations, all team members have the same development environment and can seamlessly share all information.

Flexibility/scalability

- With enterprises having to adapt, even more rapidly, to changing business conditions, speed to deliver is critical
- Cloud computing stresses on getting applications to market very quickly, by using the most appropriate building blocks necessary for deployment
- If you find that the servers you are renting are not powerful enough, you can upgrade to more powerful systems. You can add servers for short-term requirements, such as load testing.



Flexibility/scalability



Cloud Challenges/cons?

- Any ideas?

Data protection

- Data Security is a crucial element that warrants scrutiny
- Enterprises fear losing data to competition and the data confidentiality of consumers
- Moreover, the actual storage location is not disclosed, adding onto the security concerns of enterprises
- Service providers are responsible for maintaining data security and enterprises would have to rely on them

➢ Cloud computing == Your data in the computer of someone else

Data recovery and availability

- All business applications have Service Level Agreements (SLA) that are stringently followed
- Cloud providers must serve
  - Appropriate clustering and Failover
  - Data Replication
  - System monitoring
  - Maintenance
  - Disaster recovery
  - Capacity and performance management

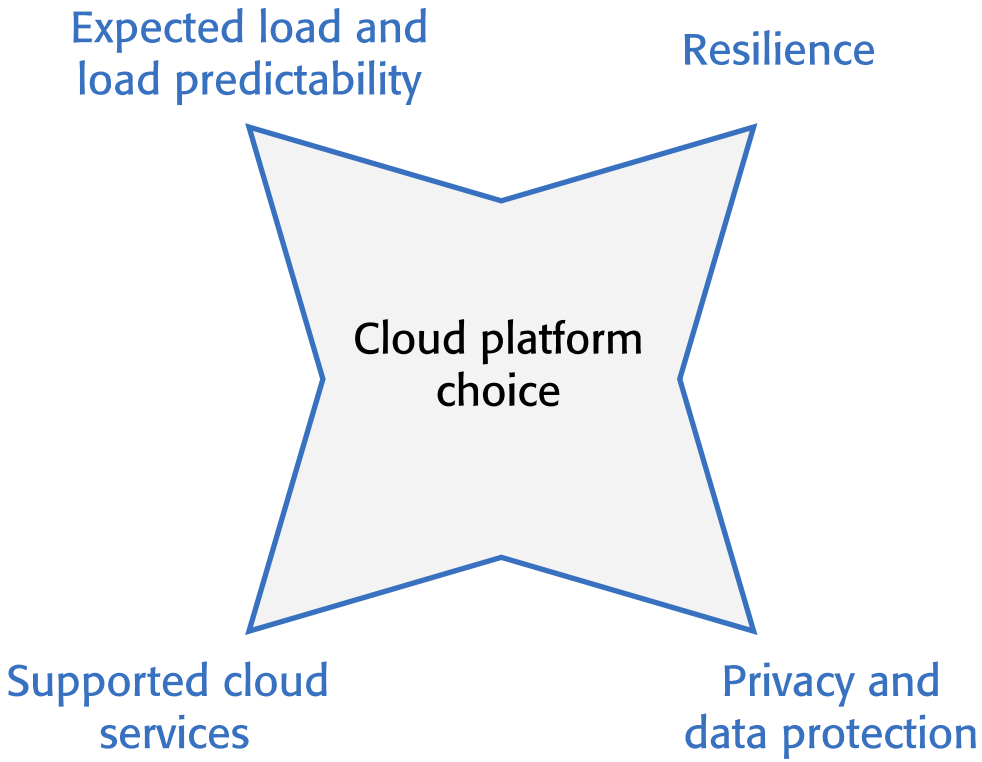
Management capabilities

- Despite there being multiple cloud providers, the management of platform and infrastructure is still a “working in progress” phase
- Features like „auto-scaling‟ for example, are a crucial requirement for many enterprises
- There is huge potential to improve on the scalability and load balancing features provided today

Regulator and compliance restrictions

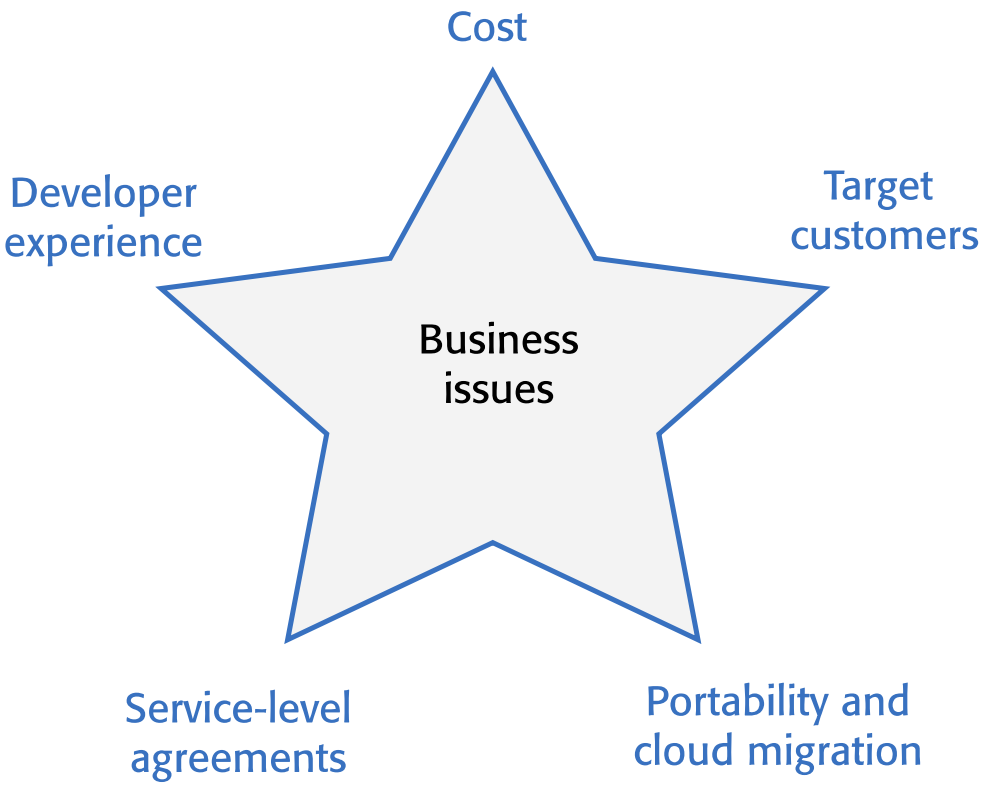
- In some of the European countries, Government regulations do not allow customer's personal information and other sensitive information to be physically located outside the state or country
- In order to meet such requirements, cloud providers need to setup a data center or a storage site exclusively within the country to comply with regulations.
- Having such an infrastructure may not always be feasible and is a big challenge for cloud providers

Figure 5.16



Technical issues in cloud platform choice

Figure 5.17



Business issues in cloud platform choice

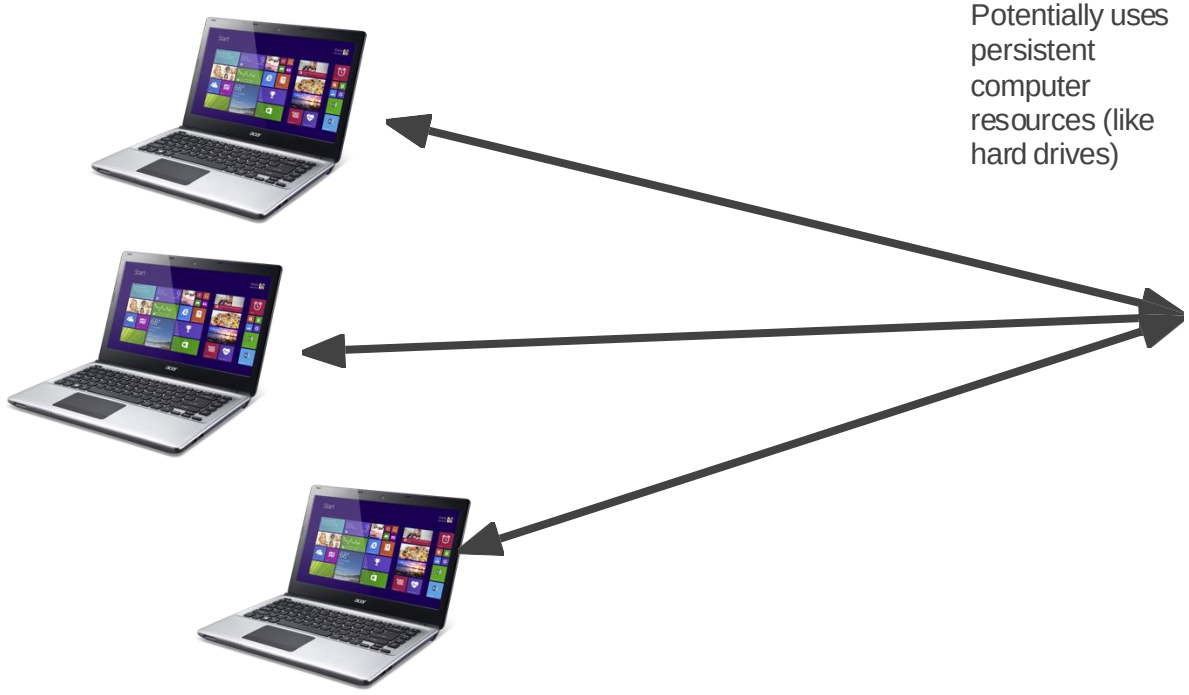
Serverless Computing Overview

What is Serverless Computing?

- Computing without servers?
- Running applications without the need to manage servers?
- Running functions instead of containers/VMs?
- Infinite scaling?
- The truth: no clear, agreed definition, i.e., no one really knows

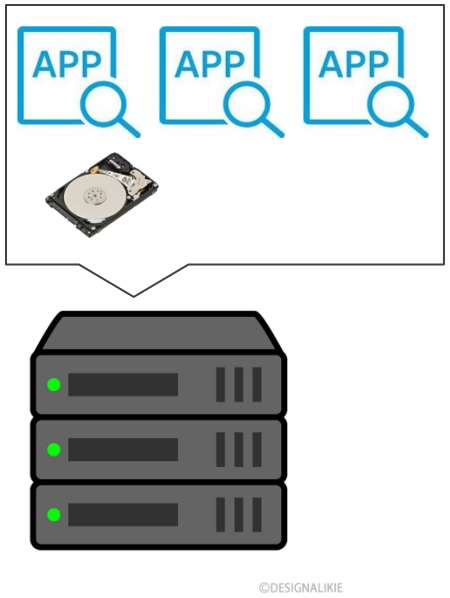
Server vs. Serverless

Traditional servers work like this:



Potentially uses persistent computer resources (like hard drives)

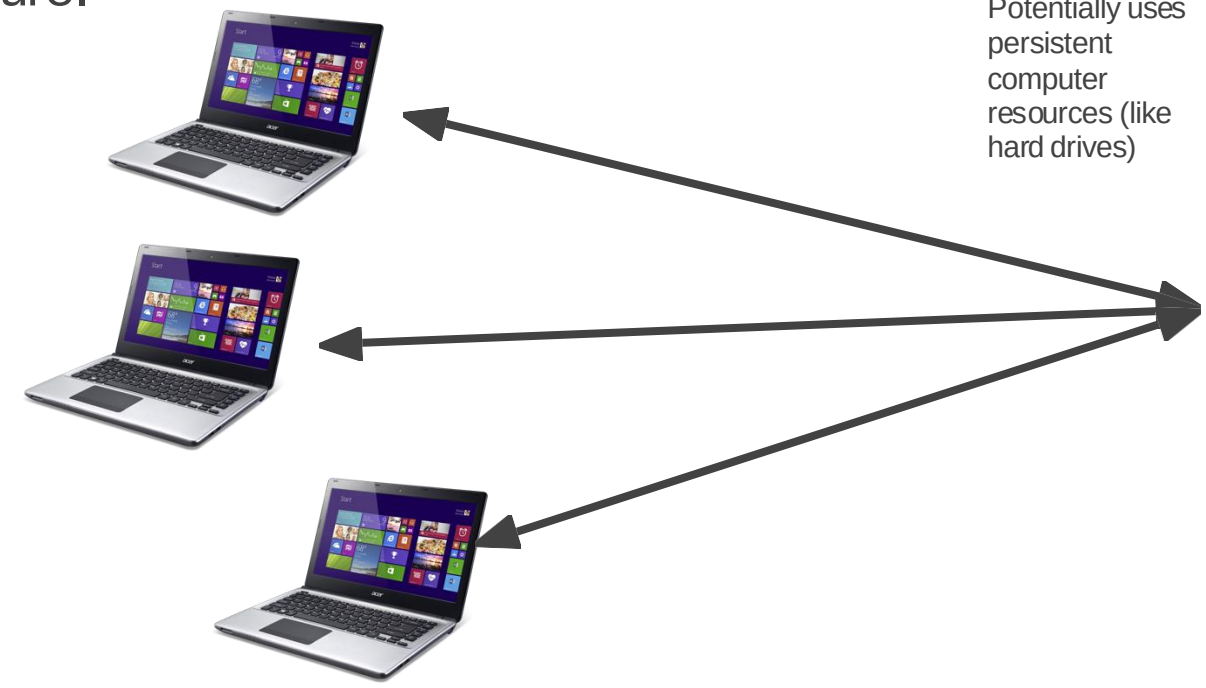
Inside: many different apps potentially working in tandem



One single (or a cluster of) recognizable, consistent servers. **Manually configured.**

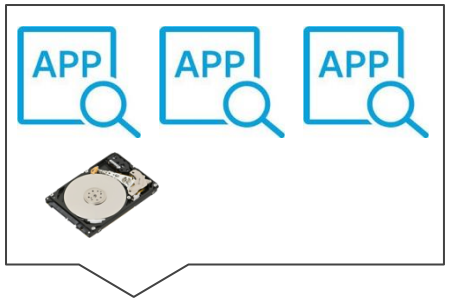
Server vs. Serverless

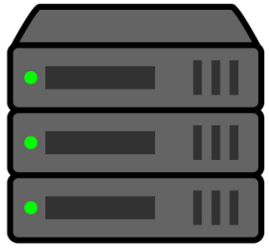
This can end up bloated, or with single points of failure.



Potentially uses persistent computer resources (like hard drives)

Inside: many different apps potentially working in tandem

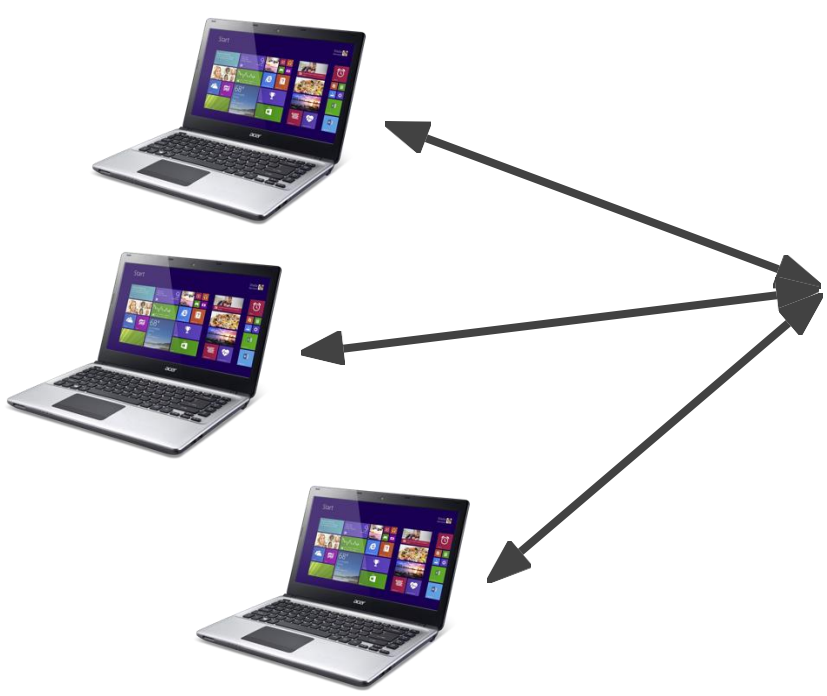




One single (or a cluster of) recognizable, consistent servers. **Manually configured.**

A Less Centralized Approach

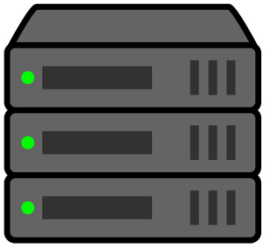
Instead, we can imagine decoupling everything.



Functions (not full applications!) – parts of your app that can run **independently**

^^Literally any computer connected to Amazon's network

Inside: only applications that need persistent storage



©DESIGNALIKIE

One single (or a cluster of) recognizable, consistent servers (backend only)

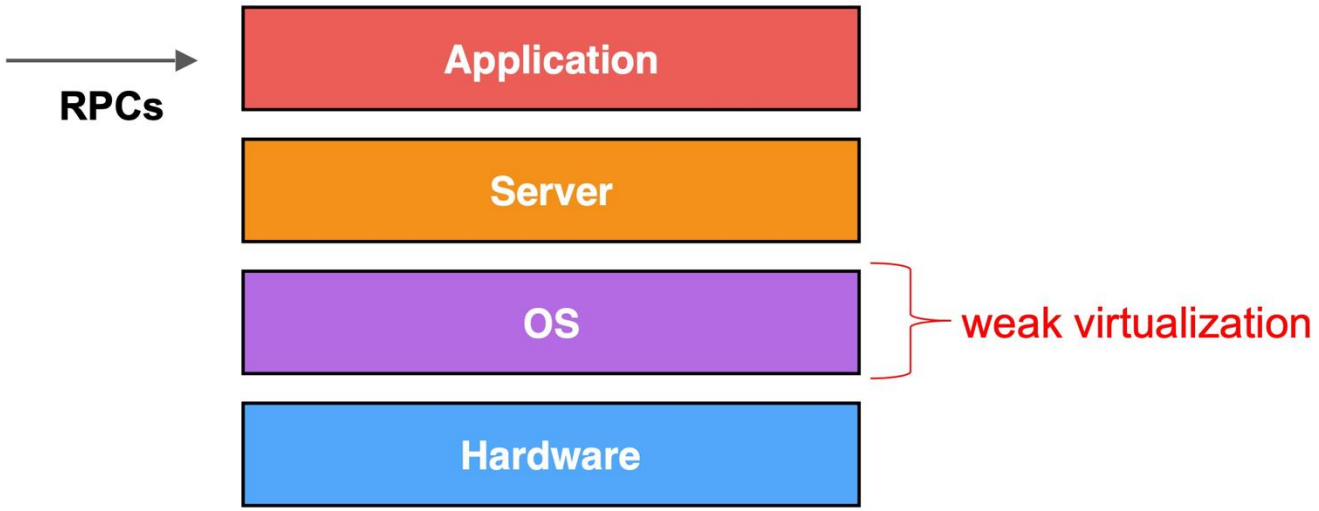
Servers vs Serverless

- Traditional servers run **software**.
  - → Usually the software **directly interacts with the network** (if it talks to the network)
  - → Requires **configuration** and **server administration**
  - → Software is **always running** to respond to events
- Traditional servers **run consistently on the same (set of) machine(s)**

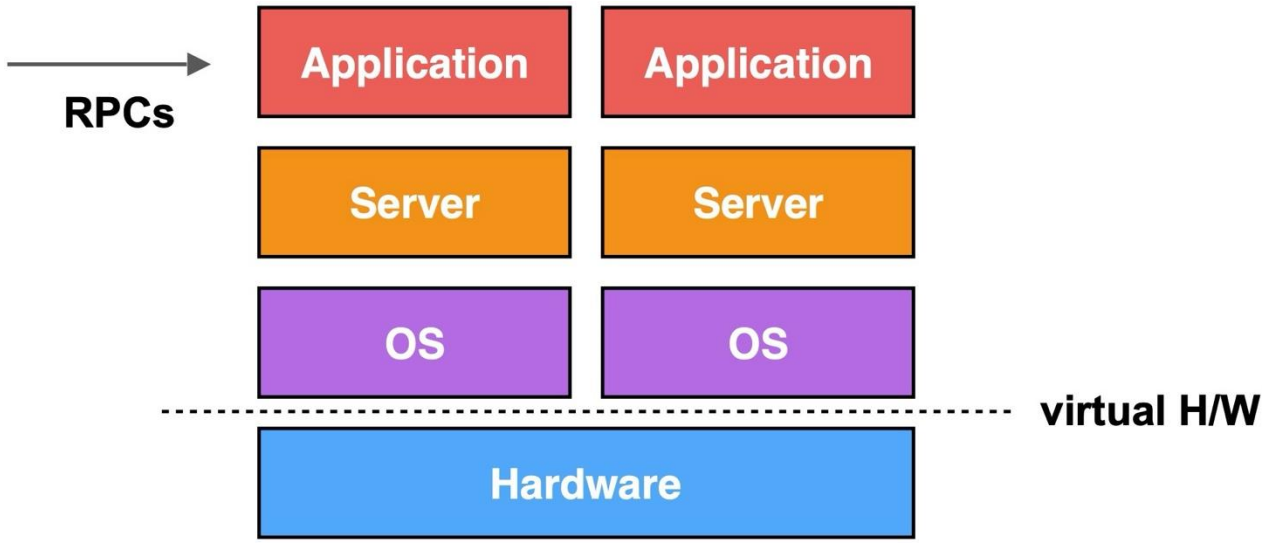
- Serverless systems run **functions**.
  - → The functions have a defined language, input, and output.
  - → Functions are **only invoked when needed** (e.g. on request)
  - → The software needs to be **written to take advantage of that system**
  - → Aside from that, **very little configuration**
- Serverless functions **are ephemeral**
  - → short-lived (max runtime ~10 seconds)
  - → cannot store files/etc
- Serverless functions **run anywhere** (no consistent machine).
  - There are still servers involved, you just don't know which ones/the scale is "infinite"
  - Usually charged per invocation

One Perspective: How Cloud and Virtualization Evolved

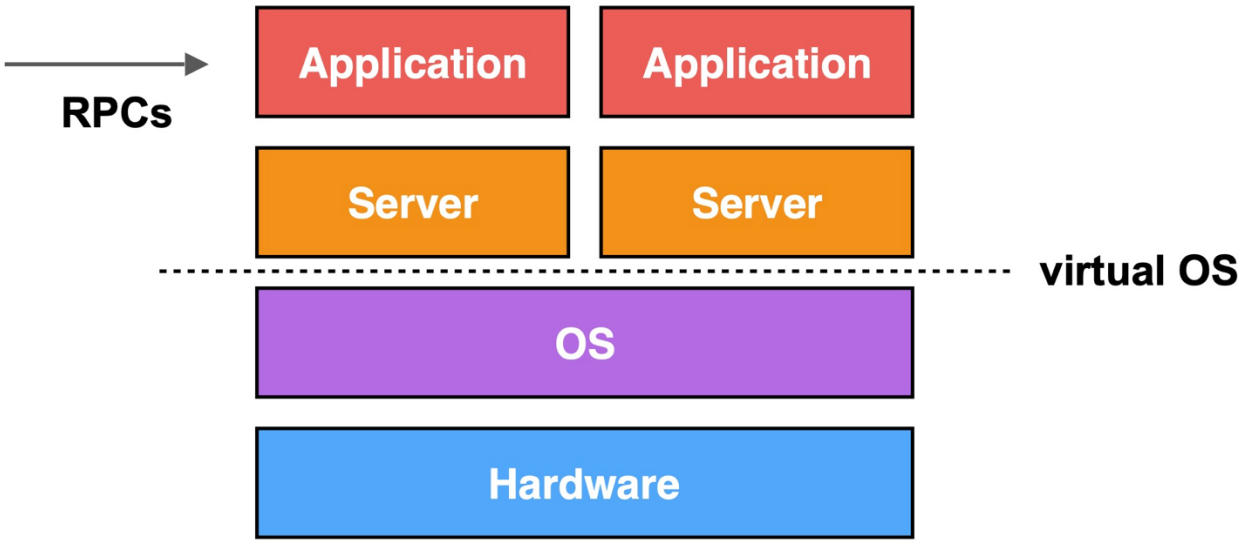
Classic Web Stack



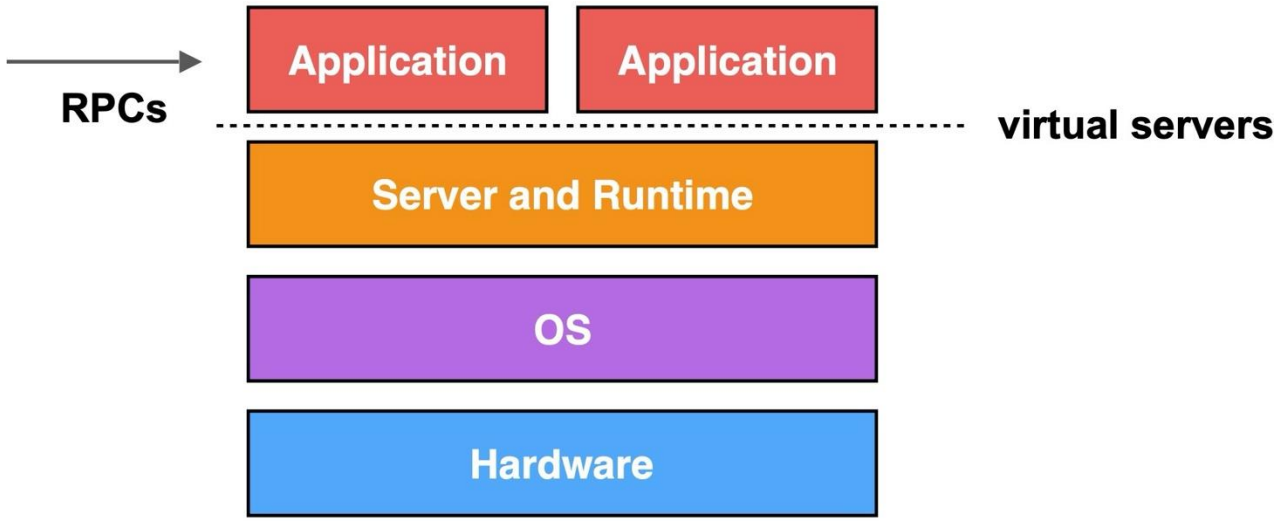
1st Generation: Virtual Machines



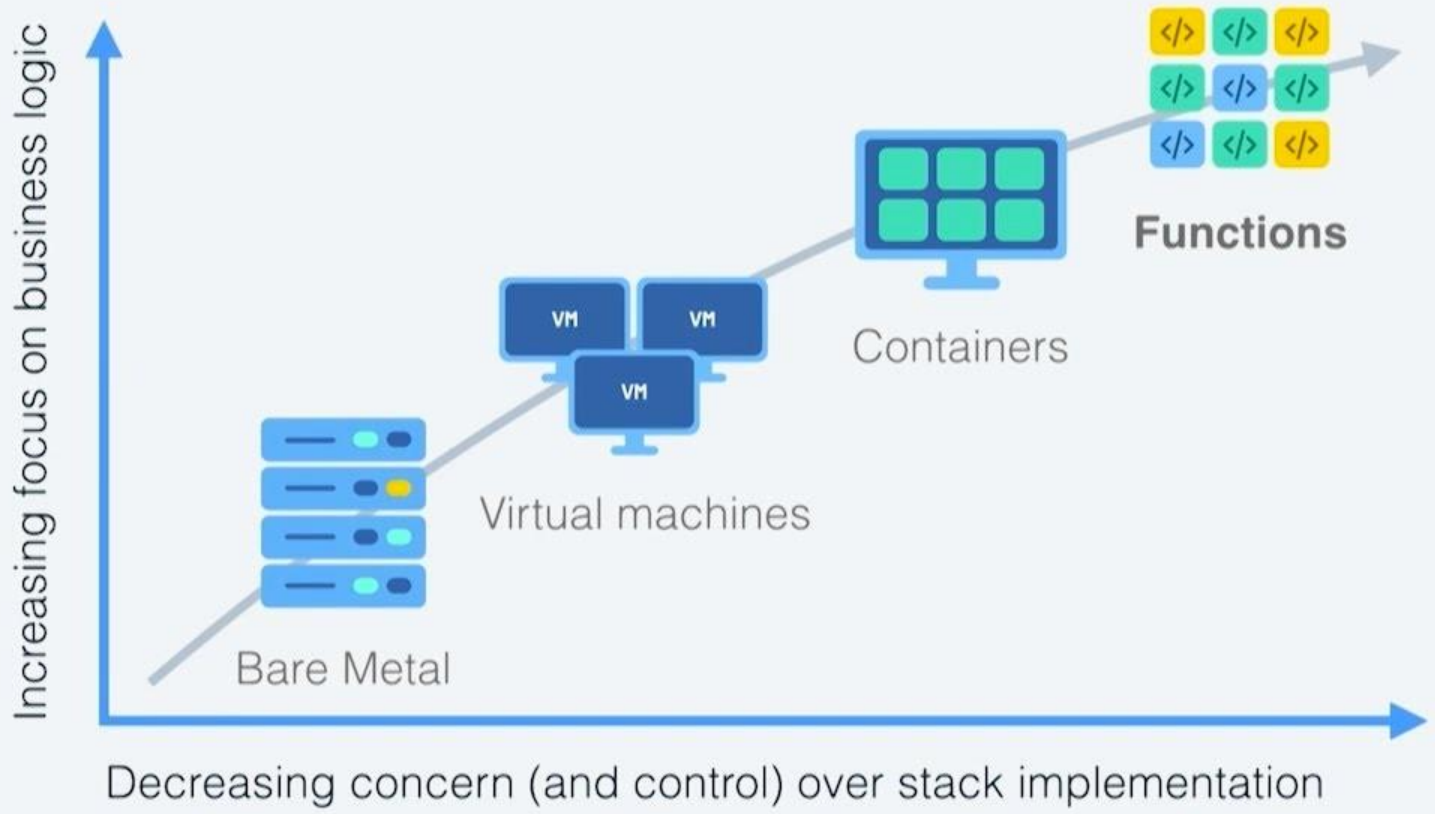
2nd Generation: Containers



3rd Generation: Serverless Computing



Evolution of serverless



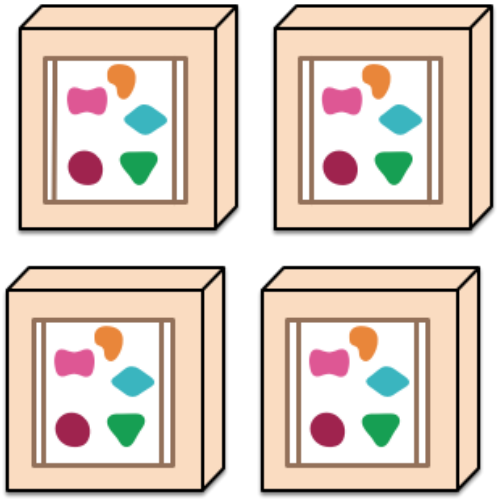
A Related Topic: Microservice

- A software architecture that develops an application as a suite of small services, each of which can be deployed and scaled independently
- When one (micro)service is in large demand, can scale it up
- Different (micro)services can be written and managed by different teams
- Changing one (micro)service will not affect the others

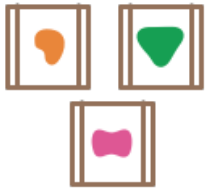
A Related Topic: Microservice

A monolithic application puts all its functionality into a single process...

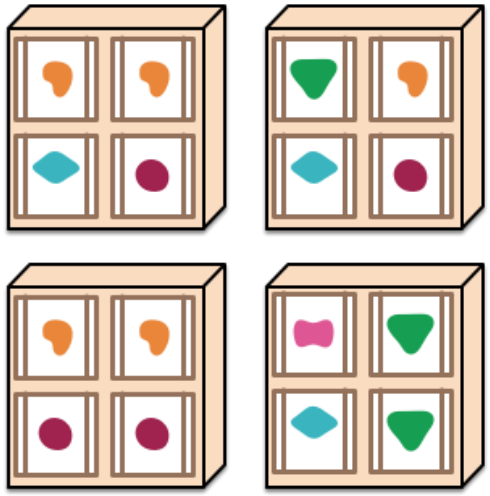
... and scales by replicating the monolith on multiple servers



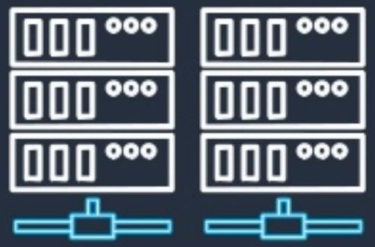
A microservices architecture puts each element of functionality into a separate service...



... and scales by distributing these services across servers, replicating as needed.



Serverless means ...



No server or container management

Flexible scaling

High availability

No idle capacity

aws

What is the essence of “Serverless Computing”? Or, what do people really like about it?

- Management-free
  - No need to handle creation, failure, replication, etc.
- Autoscaling
  - Spin up/down functions quickly based on load
- Only pay for what you use

“ I didn’t have to worry about building a platform and the concept of a server, capacity planning and all that “yak shaving” was far from my mind... However, these changes are not really the exciting parts. The killer, the gotcha is the billing by the function...

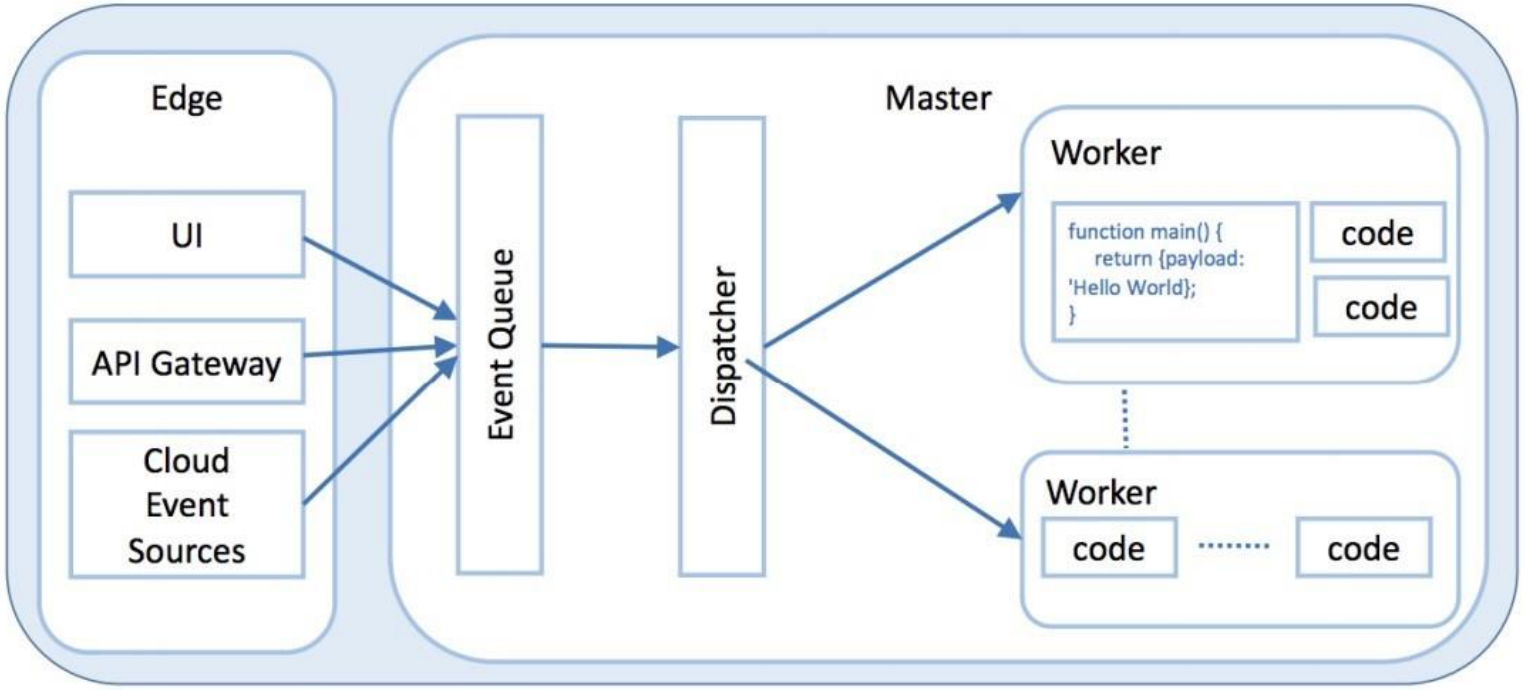
This is like manna from heaven for someone trying to build a business. Certainly I have the investment in developing the code but with application being a variable operational cost then I can make a money printing machine which grows with users…"

source: https://hackernoon.com/why-the-fuss-about-serverless-4370b1596da0

What is Today's Serverless Computing Like?

- Largely offered as Function as a Service (FaaS)
  - Cloud users write functions and ship them
  - Cloud provider runs and manages them
- Still runs on servers
- Have attractive features but also many limitations

Basic Architecture



First serverless app: BigQuery

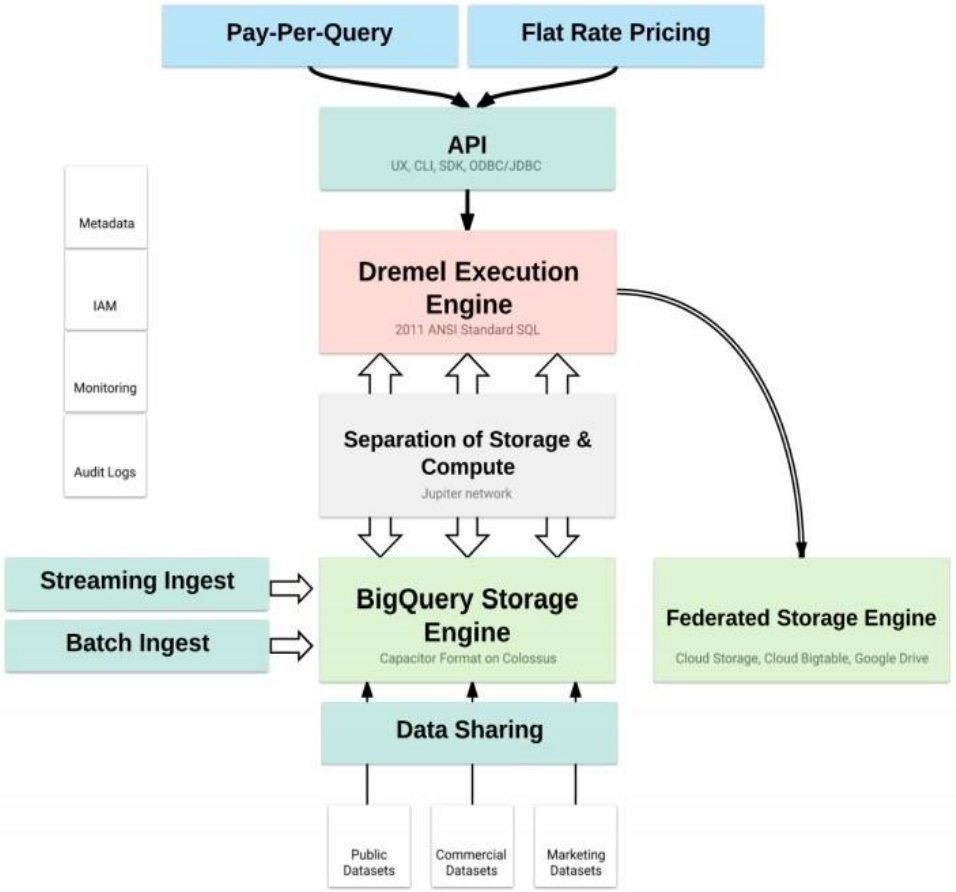
Google Big Query

Fully managed Data Warehouse

- "Arbitrarily" large data and queries
- Pay per bytes being processed
- No concept of cluster

Other similar systems

- AWS Athena
- Snowflake
- …



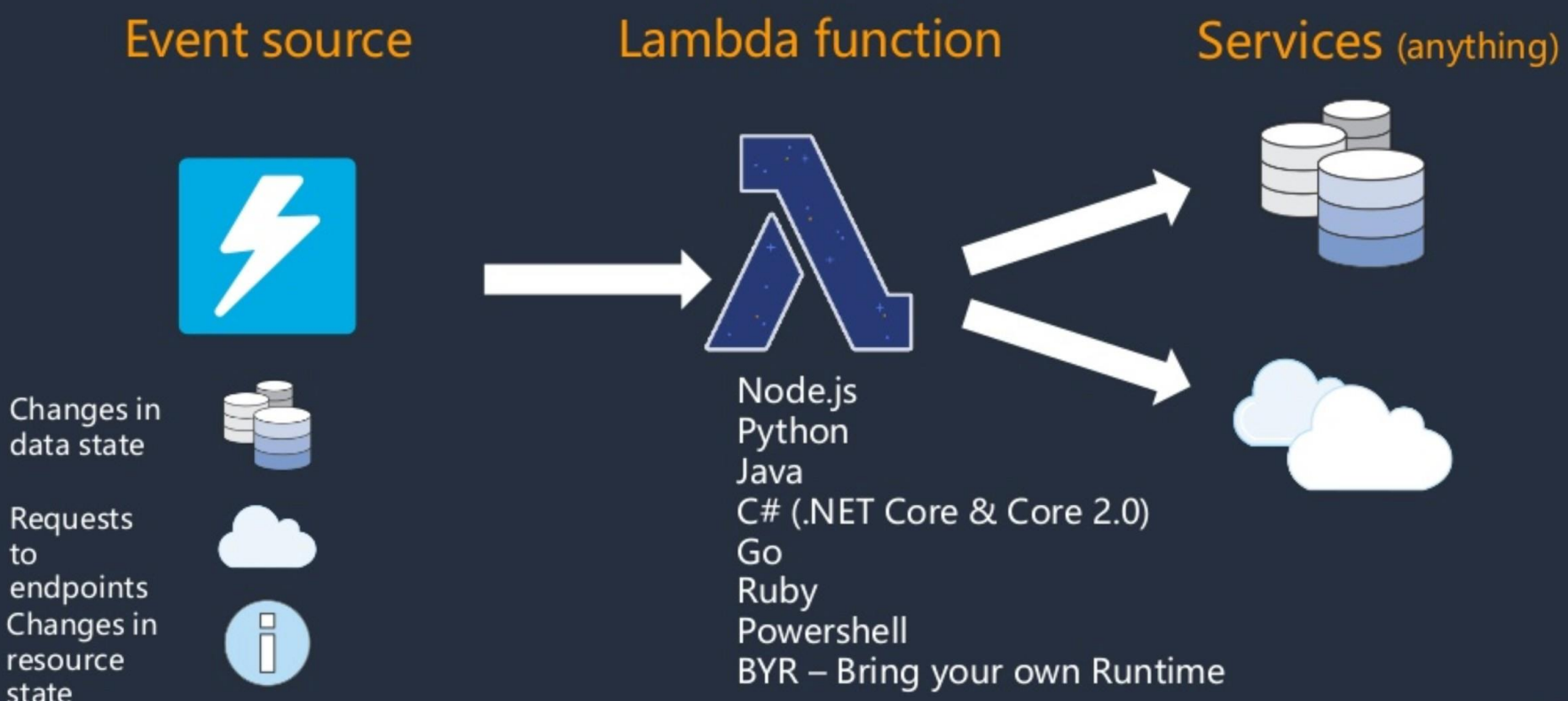
AWS Lambda

- An event-driven, serverless computing FaaS platform introduced in 2014
- Functions can be written in Node.js, Python, Java, Go, Ruby, C#, PowerShell
- Each function allowed to take 128MB - 3GB memory and up to 15min
- Max 1000 concurrent functions
- Connected with many other AWS services

Lambda Function Triggering and Billing Model

- Run user handlers in response to events
  - web requests (RPC handlers)
  - database updates (triggers)
  - scheduled events (cron jobs)
- Pay per function invocation
  - No charge when no functions run (no triggering event)
  - Billed by duration of function, configured memory size, and # of functions
    - charge *actual_time * memory_cap*

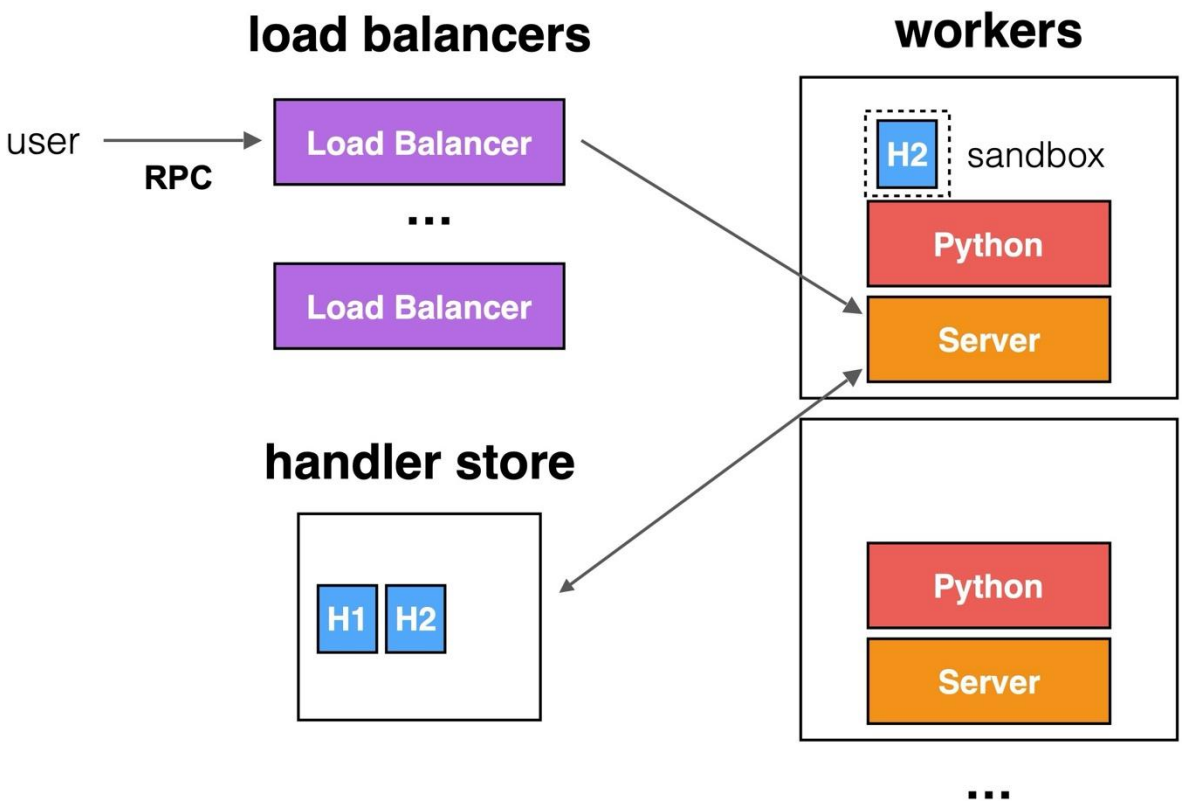
Serverless Applications



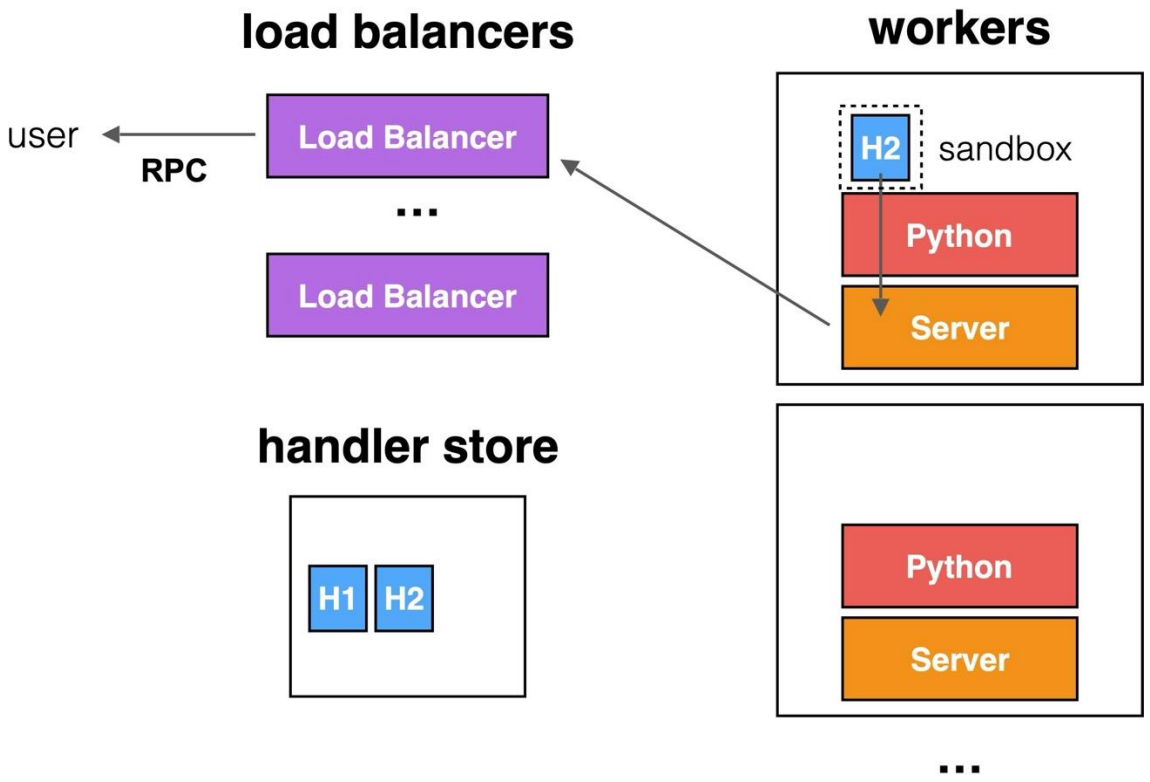
Internal Execution Model

- Developers upload function code to a *handler store* (and associate it with a URL)
- Events trigger functions through RPC (to the URL)
- Load balancers handle RPC requests by starting *handlers* on *workers*
  - Calls to the same function are typically sent to the same worker(s)
- Handlers sandboxed in containers
  - AWS Lambda reuses the same container to execute multiple handlers when possible

Execution Model



Execution Model



Azure Functions

- Debuted in 2016
- Support Python, C#, Java, JavaScript and PowerShell
- Three plans:
  - Consumption, Premium, and Dedicated
  - Dedicted: functions run on dedicated VMs and scaled manually (unlimited duration)
  - Max duration for consumption and premium: 10min and 60min
  - Max memory: 1.5GB - 14GB (depending on plans)

Google Cloud Functions

- Support Node.js, Python, and Go
- Memory size 128MB - 2GB
- Max function duration 9min
- Max number of functions per project: 1000
- Bill CPU and memory separately

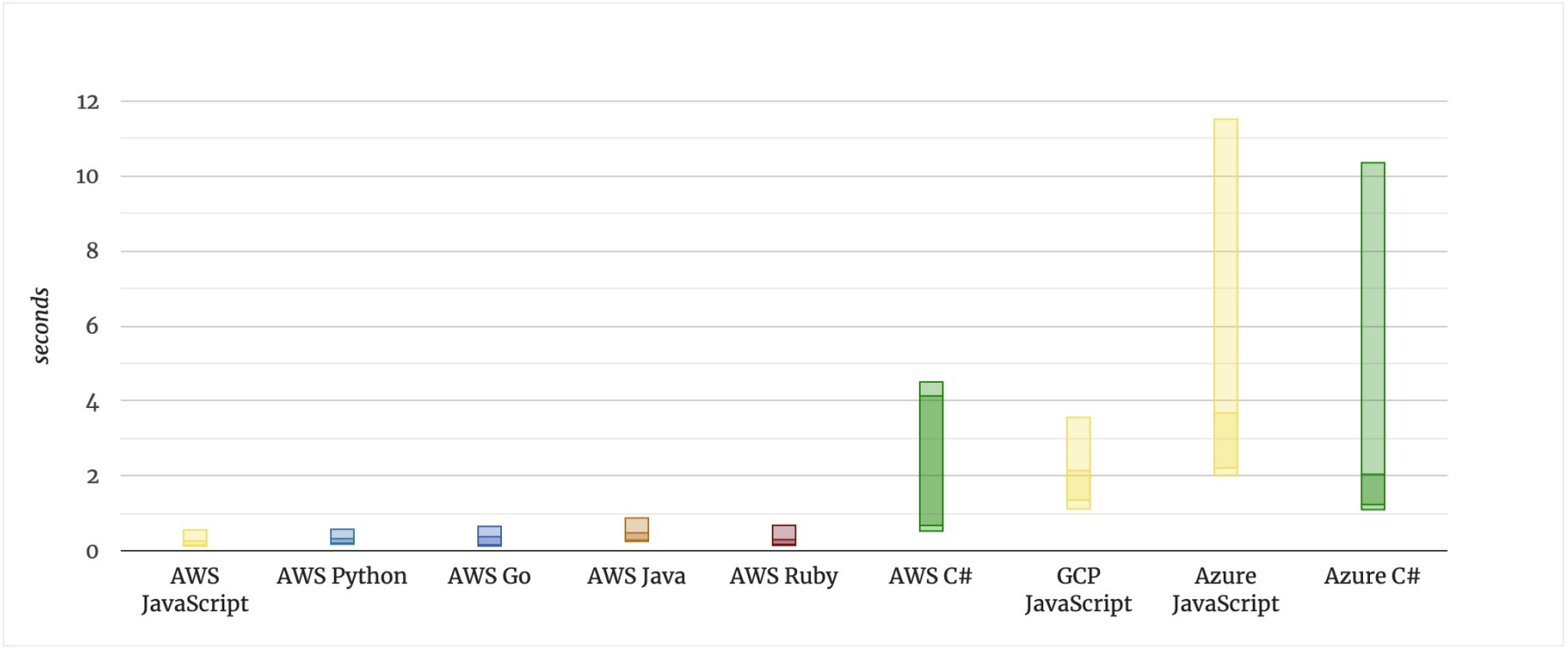
Open-Source Projects

- OpenWhisk
  - Originally developed by IBM and the core technology behind IBM Cloud Functions
- OpenLambda
  - Research prototype developed at University of Wisconsin, written in Go and based on Linux containers

Limitations of Today's Serverless Offerings

- Difficult and slow to manage states
  - Have to use (slow) cloud storage!
- No easy or fast way to communicate across functions
  - Have to go through cloud storage or other services
- Functions can only use limited resources
- No control over function placement or locality
  - e.g., starting functions on "cold" machines can be slow
- Billing model does not fit all needs

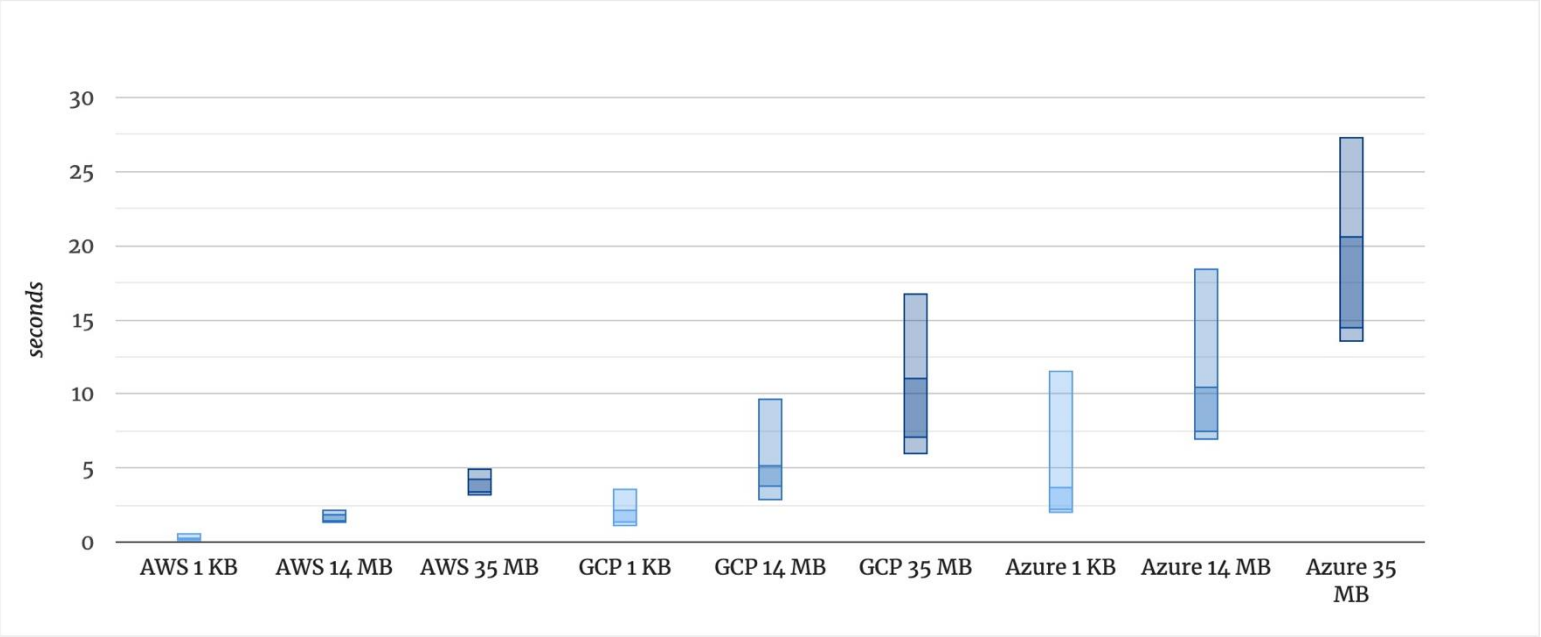
Cold Start Time of Different Languages/Offerings



Typical cold start durations per language

source: https://mikhail.io/serverless/coldstarts/big3/

Cold Start Time with Different Package Sizes



Comparison of cold start durations per deployment size (zipped)

source: https://mikhail.io/serverless/coldstarts/big3/

Good and Bad Use Cases

- Some good ones:
  - Parallel, independent, stateless tasks
  - Orchestrating functions
  - Function composition
- Some bad ones:
  - Stateful applications
  - Distributed applications and protocols
  - Applications that demand more resources (esp. memory)

Serverless ♥ Servers

- Often **servers + serverless will work together!**

Example:

- Use a **server** to host a database
- Use **serverless functions** to host a website
- Use **serverless functions** to provide APIs to request downloading music
  - Adds a download request to the database
- Use a **server** to download, encode, and upload media (using ffmpeg)
- Use **serverless functions** to retrieve information from the database, (like where the music ended up).

Various Serverless Providers

Built on top of/as a part of cloud platforms:

- AWS Lambda
- GCP Serverless
- Azure Serverless
- DigitalOcean Functions

Their own services (usually designed for website hosting-esque tasks):

- Vercel (runs on AWS Lambda)
- Netlify
- Cloudflare

Cloud resources

- AWS
- Azure
- Google Cloud
- https://www.bu.edu/tech/services/teaching/cloud-for-students/

Key points 1 (1 of 2)

- The cloud is made up of a large number of virtual servers that you can rent for your own use. You and your customers access these servers remotely over the internet and pay for the amount of server time used.
- Virtualization is a technology that allows multiple server instances to be run on the same physical computer. This means that you can create isolated instances of your software for deployment on the cloud.
- Virtual machines are physical server replicas on which you run your own operating system, technology stack and applications.

Key points 1 (2 of 2)

- Containers are a lightweight virtualization technology that allow rapid replication and deployment of virtual servers. All containers run the same operating system. Docker is currently the most widely used container technology.
- A fundamental feature of the cloud is that ‘everything’ can be delivered as a service and accessed over the internet. A service is rented rather than owned and is shared with other users.

Key points 2 (1 of 2)

- Infrastructure as a service (IaaS) means computing, storage and other services are available over the cloud. There is no need to run your own physical servers.
- Platform as a service (PaaS) means using services provided by a cloud platform vendor to make it possible to auto-scale your software in response to demand.
- Software as a service (SaaS) means that application software is delivered as a service to users. This has important benefits for users, such as lower capital costs, and software vendors, such as simpler deployment of new software releases.

Key points 2 (2 of 2)

- Multitenant systems are SaaS systems where all users share the same database, which may be adapted at run-time to their individual needs. Multi-instance systems are SaaS applications where each user has their own separate database.
- The key architectural issues for cloud-based software are the cloud platform to be used, whether to use a multitenant or multi-instance database, the scaleability and resilience requirements, and whether to use objects or services as the basic components in the system.

Next week

- Mid-semester project updates

Each team must present their product and mid-semester state. The presentation must include:

- Brief team Introduction including responsibilities.

- Using Moore's vision template, present your product vision. Briefly discuss how and why you selected this vision and your information sources. Be prepared to justify your vision.

- Present your personas and scenarios. Be prepared to justify them.

- Present your product's requirements. The requirements should be divided into priorities; must-haves, want-to-haves, nice-to-haves. The must-haves should define your MVP (Minimal Viable Product). Each requirement must be presented in the form of a job story. Note that we will be looking to see how each requirement relates to a persona and scenario. You may use either or both formats presented in class:

As a <role>, I <want | need> to <do something>

or

As a <role> I <want | need> to <do something> so that <reason>

Present and justify your high-level architecture. You may use block diagrams or UML showing components and relationships as well as database design (ERD diagrams). You should also show your UI (wireframes, hand drawings, etc.) that you're creating.

Present risks to developing your product and paths to overcome or go around. Consider the time and resources (skill set of your team) you have. Each team is lacking in some resources (time, experience, skills, etc.). What will you do about them?

Demo what you have working.

Be prepared to answer questions from Ron and Camie and other class members.

Each team member must participate in the presentation.

You must submit your slide deck and check in your code and any other artifacts you've created. Be sure to submit links to your Git repository, giving access to Camie and me and any other online resource you're using.

Each team will have 10 minutes MAX including questions.

Grading Rubric:

- Persona, scenario, and product requirements and how they are relevant to each other. (30 points).

- Presentation and demo quality. Please be sure to have your presentation well thought out. We're not looking for a finished product but want to know what you've accomplished so far. Please be prepared. (30 points).

- High level architecture including code, database, UI, etc. (20 points).

- Risk analysis and remediation plans. (10 points).

- Good use of resources including team responsibilities. Is everyone contributing? (10 points).

Copyright